ZF-Microglia-AI — Software Guide

A developer/technical reference: every module, every function, what algorithm it implements, and how the pieces connect. Companion to GUIDE.md (user-facing, how to operate the plugin) and STATISTICS_GUIDE.md (the statistics formulas in detail — this guide summarizes them, that one derives them). Written for someone reading the source for the first time, or citing the pipeline's algorithms in a thesis/paper.

Pseudocode blocks use a plain, language-agnostic notation (not literal Python) — `for`/`if`/`while`, indentation for scope, ← for assignment where it helps distinguish from equality, and function calls written `name(args)`. Real function/parameter names are used throughout so a reader can go from pseudocode straight to `grep` in the real source.

Table of Contents

1. Architecture overview
2. Common infrastructure — file I/O, secrets, GPU detection, live-progress capture, generic coarse-to-fine sweeping, `_widget.py`'s own shared helpers
3. Stage 1 — Skin removal — `_inference.py`, `_background.py`
4. Stage 2 — Label creation — `_labeling.py` (Pixel Classifier), `_cellpose_seg.py` (Cellpose-SAM), `_sanding.py`
5. Stage 3 — Label editing & correction — `_labeling.py` (editing tools), `_grow_correct.py`, `_contrast_sweep.py`, `_auto_correction.py`
6. Stage 4 — Statistics — `_statistics.py`
7. Stage 5 — AI Tools — `_gt_annotation.py`, `_xzyz_patches.py`, `_crop_truncation.py`, `_combine_crops.py`, `_branch_calibration.py`, `_training_jobs.py`
8. Stage 6 — Sweeps & Utilities — `_sieve.py`, `_brain_sweep.py`, `_pixel_sweep.py`, `_krendl_sweep.py`, `_epoch_sweep.py`, `_gt_score.py`, `_gt_package.py`, `_gt_toolkit.py`
9. `_widget.py` — application shell

10. Module reference (quick index)

---

1. Architecture overview

What the plugin does, as a pipeline

```
raw confocal stack (.tif/.ims)
        │  Stage 1 — Skin Removal (_inference.py, _background.py)
        ▼
brain_only volume (_ExtRm / _NoBG / _RndFill)
        │  Stage 2 — Label Creation (_labeling.py or
_cellpose_seg.py)
        ▼
Labels layer (one integer ID per cell)
        │  Stage 3 — Label Editing & Correction (_labeling.py,
_grow_correct.py,
        │            _contrast_sweep.py, _auto_correction.py)
        ▼
corrected Labels layer
        │  Stage 4 — Statistics (_statistics.py)
        ▼
per-cell CSV (shape, branching, spatial, intensity, optional LLM
description)
```

Two more subsystems sit alongside this straight-line pipeline, not inside it:

- **Stage 5 — AI Tools**: builds and trains the two models Stages 1 and 2 depend on (MONAI for skin removal, Cellpose-SAM for label creation), plus the ground-truth annotation tool that produces the training data in the first place.
- **Stage 6 — Sweeps & Utilities**: GT-verification tools that check a pipeline parameter against real ground truth, and the GT Toolkit Tuning Tool that runs them for one fish and (only when the GT is marked verified) recalibrates the shared values.

Where state lives

There is no persistent application database. Three places hold state:

1. **napari's own layer list** (`self._viewer.layers`) — the volumes and label arrays themselves, as `napari.layers.Image`/ `napari.layers.Labels` objects. Every pipeline stage reads its input

from a layer and writes its output as a new (or in-place-mutated) layer. This is the plugin's real "shared memory" between stages.
2. **`~/.config/napari-zf-microglia-ai/config.json`** (`self._state["config"]`, read/written via `_load_config()`/`_save_cfg()` in `_widget.py`) — every slider/checkbox default, every "recommended" GT-swept value (see `_update_gt_history()`), file-browse paths, and email/API settings (except the secrets themselves — see `_secrets.py` in §2).
3. **The OS credential store or a local encrypted file** (`_secrets.py`) — the two values deliberately kept out of `config.json`: the SMTP app password and the LLM API key.

The threading pattern used by almost every "Run"/"Sweep" button

Every long operation in this plugin — MONAI inference, Cellpose-SAM's `do_3D`, a GT sweep, a training launch — follows the same shape, to keep napari's own Qt event loop responsive:

```
def _on_run_button_click():
    validate inputs; if invalid → show error, return
    disable the button; set status text to "Running..."
    result ← {}                                # plain dict, mutated
by the worker
    def _worker():
        try:
            result["value"] ← do_the_actual_work(...)   # runs on
a background thread
        except Exception as exc:
            result["error"] ← str(exc)
    thread ← self._job_thread("<tool name>", _worker, result);
thread.start()
                                               # timed job: may
email at the end (§9)

    timer ← QTimer()                           # polls on the GUI
thread
    def _poll():
        if thread.is_alive():
            if "_progress" in result: update status text
            return                             # check again next
tick
        timer.stop()
        if "error" in result: show it; re-enable button; return
```

```
        apply result["value"] to the relevant napari layer(s)   #
GUI-thread-only work
        re-enable button
    timer.timeout.connect(_poll); timer.start(200)   # re-check
every 200 ms
```

`result` is the hand-off point between the background thread (which must never touch Qt widgets or `viewer.layers` directly — napari/Qt are not thread-safe) and the GUI thread (which owns every widget and layer mutation). A `progress_cb` callback threaded through the actual algorithm lets the worker report intermediate status without touching the GUI itself — it just writes into `result["_progress"]`, which the next poll tick picks up.

Two further patterns build on this:

- **Detached processes** (`_training_jobs.py`, Stage 5): a training run is a genuinely separate OS process (`conda run ...`), not a background thread inside napari — it survives napari closing entirely. The same poll-a-shared-state loop just polls a PID + log file on disk instead of an in-process `Thread`.
- **The hang watchdog** (`_hang_watch_start`/`_hang_watch_stop` in `_widget.py`, §9): wraps the thread-start of the slowest operations (Protect Skin as Label, Auto-correct) with `faulthandler.dump_traceback_later()`, so a run that's still going after 30 minutes prints every thread's real stack to the terminal — turning "is it stuck?" into "here's exactly which line it's on."

Layer-selection philosophy

Early in this project every editing tool read its input from whatever layer happened to be "active" in napari's own layer list (`_active_labels_layer()`: active selection, else topmost Labels layer, else `None`) — implicit, and silently wrong whenever the wrong layer happened to be selected. Every tool in Stage 3 (Edit MG Labels) now instead reads from one **explicit, shared combo-box selector** (`_edit_labels_layer()`, `self._edit_signal_combo`/`_edit_labels_combo`/`_edit_mask_combo`) that the user sets once, not per tool — see §9.

---

2. Common infrastructure

Modules and helpers used across more than one pipeline stage.

`_io.py` — file loading with metadata-aware voxel scale

Every downstream algorithm that measures a physical distance (µm) needs the true voxel size (Z spacing vs. XY pixel size — these stacks are anisotropic, typically ~5.75:1). This module's only job is finding that scale and loading the raw pixel data consistently for `.tif`/`.tiff`/`.ims`.

`_calc_anisotropy(voxel_z_um, voxel_x_um, voxel_y_um)`

```
xy ← (voxel_x_um + voxel_y_um) / 2
return voxel_z_um / xy   (or 1.0 if xy ≤ 0)
```

`extract_tif_metadata(tif_path)` — reads ImageJ-embedded metadata from a TIFF's own tags (`spacing`/`finterval` for Z, `XResolution`/`YResolution` for XY, which ImageJ stores as *pixels per µm*, hence the reciprocal). Returns `None` on any failure so the caller can fall through to the next metadata source.

`parse_metadata(metadata_path)` — parses a Leica-format `*_metadata.txt` sidecar file with regexes for camera pixel width/height, Z step size, and every `TotalConsolidatedOpticalMagnification` value present (there are usually two: the objective's, ≥10×, and a digital zoom factor, <10×):

```
cam_x, cam_y ← regex-extracted "Pixel Width/Height (µm)"
voxel_z      ← regex-extracted "StepSize"
mags         ← every "TotalConsolidatedOpticalMagnification" value
found
obj_mag      ← first mag ≥ 10        (the objective)
zoom_mag     ← first mag < 10        (digital zoom)
total_mag    ← obj_mag × zoom_mag
voxel_x, voxel_y ← cam_x / total_mag, cam_y / total_mag
```

Returns `None` if any of the four regex groups is missing.

`find_best_metadata_match(file_path)` — a stack's own metadata file isn't always named exactly `<stem>_metadata.txt`; this looks for an exact match first, then the closest name (`difflib.SequenceMatcher` string-similarity ratio, threshold 0.3) among every `*_metadata.txt` in the same folder, then repeats one directory up if nothing was found (metadata sometimes lives one level above the stack).

`load_file(path)` — the public entry point:

```
if suffix is .tif/.tiff:
    raw ← tifffile.imread(path)
```

```
    metadata ← find_best_metadata_match(path) →
parse_metadata(...)
               or extract_tif_metadata(path)
               or the hardcoded default (1,1,1) µm/vox
    if raw is 3D:  channels ← [(raw, stem, metadata)]
    if raw is 4D:  channels ← [(raw[c], f"{stem}_ch{c}", metadata)
for each channel c]
elif suffix is .ims:
    f ← ImsReader(path)
    metadata ← find_best_metadata_match(...) → parse_metadata(...)
               or f's own embedded resolution
    channels ← [(f.get_Volume_At_Specific_Resolution(...), name,
metadata)
                for each channel]
return channels    # list of (volume, layer_name, metadata_dict)
```

Every channel is returned separately so the caller (`_reader.py`, or `_widget.py`'s own Open-file handler) can add each as its own napari `Image` layer, letting the user pick which channel is the actual microglia signal.

`_reader.py` — napari File-menu integration

Registers this plugin as a napari reader (npe2 `napari.yaml` "readers" contribution, `*.tif`/`*.tiff`/`*.ims`) so `.tif`/`.ims` files opened via napari's own File → Open work correctly, not just via this plugin's own Open button.

```
get_reader(path):
    if path's suffix in {.tif, .tiff, .ims}: return _read_file
    else: return None                          # tells napari "not
mine"

_read_file(path):
    channels ← _io.load_file(path)
    for each (volume, name, metadata) in channels, with index i:
        colormap ← _CH_COLORMAPS[i mod 4]       # gray, green,
magenta, cyan, repeating
        yield (volume, {name, colormap, scale: metadata["scale"]},
"image")
```

`_secrets.py` — layered credential storage

Two values need to persist between sessions without ever sitting in the plugin's own plaintext `config.json`: the SMTP app password (email notifications) and the LLM API key (Statistics' description backend). Two-tier storage:

```
get_secret(key):
    try: value ← keyring.get_password(SERVICE_NAME, key)   # Tier
1: OS credential store
         if value: return value
    except: pass
    return _fallback_get(key)                               # Tier
2: local Fernet-encrypted file

set_secret(key, value):
    try:
        keyring.set_password/delete_password(...)           # Tier
1
        _fallback_set(key, "")     # clear any stale Tier-2 copy
now that Tier 1 works
        return None                # success, nothing to report
    except Exception as tier1_exc:
        try:
            _fallback_set(key, value)                       #
Tier 2
            return None (but print a warning naming the weaker
guarantee)
        except Exception as tier2_exc:
            return an error string (both tiers failed -- caller
shows it, doesn't crash)
```

`_get_fernet()` generates a machine-local AES key on first use (`Fernet.generate_key()`), stores it `chmod 600` next to the encrypted blob — a real but deliberately weaker guarantee than Tier 1 (the key lives beside what it protects), kept because Tier 1 (`keyring`'s Secret Service backend) reliably fails on SSH-only Linux sessions with no PAM-driven keyring unlock, confirmed directly on this project's own workstation.

`migrate_plaintext_secrets(cfg)` — a one-time upgrade path: if an old `config.json` still has `api_key`/`notify_smtp_password` in plaintext (from before this module existed), move them into `set_secret()` and strip them from the dict that gets written back.

`_gpu_check.py` — GPU/VRAM classification

Checked once at import time (hardware doesn't change mid-session) and cached into module-level constants (`GPU_HAS_CUDA`, `GPU_VRAM_GB`, `GPU_MEETS_RECOMMENDED`, `GPU_NAME`, `GPU_MSG`):

```
check_gpu():
    try:
        if not torch.cuda.is_available(): return {has_cuda:
False, ...}
        props ← torch.cuda.get_device_properties(0)
        vram_gb ← props.total_memory / 1024³
        meets ← vram_gb ≥ 8
        return {has_cuda: True, vram_gb, meets_recommended: meets,
name: props.name}
    except: return {has_cuda: False, ...}     # never raises --
worst case, a cautious banner
```

Informational only — Tab 5 (AI Tools) is always shown regardless of the result; this only decides which color/text its disclaimer banner uses.

`_live_progress.py` — surfacing libraries' own hidden progress output

MONAI's `sliding_window_inference(progress=True)` draws a `tqdm` bar to raw `sys.stderr`; Cellpose routes everything through Python's `logging` module but never calls its own `logger_setup()` unless its CLI is used, so by default its log records go nowhere — not even a terminal. `capture_live_output(push)` is a context manager that makes both visible in the GUI for the duration of one call:

```
capture_live_output(push):
    if another capture is already active: yield (no-op
passthrough); return
    save real sys.stdout/sys.stderr and the root logger's level
    sys.stdout ← sys.stderr ← a _LineSink(push)   # buffers
partial writes,
                                                  # pushes a
complete line on \n OR \r
                                                  # (tqdm
redraws with \r; GUI can't
                                                  #  overwrite
in place, so \r → new line)
```

```
    attach a logging.Handler to the root logger that calls
push(formatted record)
    set root logger level to INFO if it was less verbose
    yield                                          # caller's real
work runs here
    finally: restore stdout/stderr, remove the handler, restore
the log level
```

`push` must be thread-safe (the caller always passes the same `result["_progress"] = msg` pattern every other worker uses).

`_sieve.py` — generic coarse-to-fine sweep narrowing

Shared by every GT-sweep tool that has one continuous "primary axis" (MONAI Threshold, BG Threshold, Smooth σ XY, Cellprob) — born from a real manual workflow (sweep a wide range coarsely, narrow around the winner, narrow again) generalized into one reusable engine that knows nothing about what it's sweeping.

```
sieve_grid(lo, hi, center, half_width, step):
    stage_lo ← max(lo, center - half_width)
    stage_hi ← min(hi, center + half_width)
    if stage_hi ≤ stage_lo: return [clip(center, lo, hi)]     #
degenerate range → 1 point
    return every value from stage_lo to stage_hi, step apart

run_sieve(run_stage_fn, lo, hi, coarse_step, refine_steps,
progress_cb, cancel_check):
    stage_specs ← [(None, None)] + refine_steps    # stage 1 =
full range, no narrowing
    values ← grid from lo to hi, step = coarse_step
    for each (half_width, step) in stage_specs, with index i:
        if cancel_check(): mark cancelled; break
        if i > 0: values ← sieve_grid(lo, hi, best_value,
half_width, step)
        best_value, stage_result ← run_stage_fn(values)   # caller
runs ONE 1D grid,
                                                            #
everything else held fixed
        record this stage (values, best_value, stage_result)
        if stage_result says cancelled: mark cancelled; break
    return {stages: [...], final_best_value, final_result,
cancelled}
```

`run_stage_fn` is the only thing that actually knows about MONAI/Cellpose/etc. — it's a closure the caller builds, e.g. "run the BG-Threshold sweep at these candidate values, holding Erosion fixed at its current slider value, return the winning threshold and the sweep's own report dict."

`_widget.py`'s own shared helpers

A handful of small functions used throughout `_build_ui()` (the ~4,200-line method that constructs every tab — see §9), not tied to any one pipeline stage:

- **`_add_reliable_spinbox(row_layout, slider, min, max, step, decimals=None)`** — every numeric slider in this plugin is paired with a real `QSpinBox`/`QDoubleSpinBox` synced bidirectionally (`slider.valueChanged → spin.setValue`, `spin.valueChanged → slider.setValue`), replacing `superqt`'s own built-in numeric label whose width reset unpredictably on first show. Returns the spinbox.
- **`_set_layout_widgets_visible(layout, visible)`** — recursively walks a `QVBoxLayout`/`QHBoxLayout` tree (rows mix `addWidget` and nested `addLayout` calls) and calls `setVisible()` on every real widget found, since only widgets (not layouts) support it directly.
- **`_make_collapsible(groupbox, start_expanded=True)`** — turns any already-built `QGroupBox` into a collapsible section: makes its own title bar checkable, and toggling it calls `_set_layout_widgets_visible` on the group's contents. Most group boxes go through this; Tab 2's Common Settings and Tab 3's selector, Correction Settings, Sanding and Contrast-calibration groups don't (always open).
- **`_wrap_scroll(widget)`** — wraps a fully-built tab page in a vertical-only `QScrollArea` (`setWidgetResizable(True)`, horizontal scrollbar forced off) so a tab taller than the napari window stays fully reachable.
- **`_add_recommended_label(layout, initial, unit, noun)`** — the read-only "Recommended: X" line under a GT-sweepable slider, kept separate from the slider's own editable value so a GT-confirmed sweep's finding is never silently overwritten by the user nudging the slider to test something else.
- **`_add_gt_checkbox(layout, hint, visible=True)`** — the "This is verified ground truth" one-shot opt-in: off by default, un-checked by the tool's own poll after every run, and the only thing that lets a sweep move a shared recommended value (Min volume, a slider's own "Recommended" line, etc.) rather than just reporting a result.

> Built with `visible=False` for the four sweeps folded into the GT Toolkit Tuning Tool (MONAI, BG Threshold, Sigma, Cellprob) and for Calibrate branch_radius — those boxes exist but are only ever ticked programmatically by `_gtk_launch_step()`; Verify Best Epoch keeps a visible one.

---

3. Stage 1 — Skin removal

UI: Tab 1 — Skin Remover. **Modules**: `_inference.py` (MONAI brain segmentation), `_background.py` (three background-removal modes). **Goal**: given a raw confocal stack, produce `brain_only` — everything but the brain zeroed — for Stage 2 to label.

`_inference.py` — MONAI 3D U-Net sliding-window inference

`_normalize(volume)` — percentile normalization, the same transform the model was trained under:

```
p01, p99 ← 1st and 99th percentile of volume
return clip((volume - p01) / max(p99 - p01, 1), 0, 1)
```

`predict_probability(volume, model_path, device)` — the expensive half of inference, split out on its own so a GT-sweep tool can run it once and cheaply re-threshold many candidate values afterward (see §8):

```
model ← UNet(3D, 1→1 channel, channels=(32,64,128,256,512), 2 res-
units/level)
        load weights from model_path checkpoint; eval mode
volume_t ← normalize(volume) as a (1,1,Z,Y,X) tensor on device
pred_logits ← sliding_window_inference(volume_t, roi=(64,192,192),
                  batch=4, overlap=0.25, mode="gaussian",
predictor=model)
              # slides a 64×192×192 window across the whole
volume, blending
              # overlapping predictions with a Gaussian-weighted
average
              # (softer seams than a hard-edge tile stitch)
return sigmoid(pred_logits)     # per-voxel probability, 0..1
```

`postprocess_probability(pred_prob, threshold)` — the cheap half, safe to call repeatedly per candidate threshold:

```
raw_mask ← pred_prob > threshold
labeled, n_components ← connected_components(raw_mask)
clean ← the single largest component            # drops isolated
blobs / mislabeled tissue
clean ← fill_holes(clean)                         # solid,
contiguous brain
return clean as uint8   # "brain_mask", always un-eroded
```

`run_inference(volume, model_path, threshold, device, erosion_voxels)` — glues the two together and applies the optional erosion:

```
pred_prob ← predict_probability(...)
brain_mask ← postprocess_probability(pred_prob, threshold)     #
always un-eroded — this is

# what gets saved as brain_mask.tif
if erosion_voxels > 0:
    eroded_mask ← binary_erosion(brain_mask,
iterations=erosion_voxels)
else:
    eroded_mask ← brain_mask
brain_only ← volume × eroded_mask       # zero everything the
(possibly eroded) mask excludes
return brain_mask, brain_only, eroded_mask
```

`eroded_mask` is the one downstream background-removal steps must use as "the brain boundary" — using `brain_mask` there instead would silently discard the Erosion slider.

`_background.py` — background estimation and removal, 3 modes

All three modes share one background estimate and one threshold rule:

`_estimate_background(volume, brain_mask)`:

```
brain_pixels ← volume[brain_mask]                    # only INSIDE
the (possibly eroded) brain
hist ← 1000-bin histogram of brain_pixels
bg_mode ← the histogram bin center with the highest count   # the
population mode, robust
                                                              #
to a skewed intensity tail
```

```
bg_values ← every brain_pixel ≤ bg_mode
return bg_values, bg_mode, bg_mode, bg_mode          # (median/
min/max all collapse to bg_mode
                                                        #  in the
current implementation)
```

`_threshold(volume, brain_mask, tolerance_pct)`:

```
bg_max ← _estimate_background(...)'s bg_mode
tol ← (max(volume) - min(volume)) × tolerance_pct / 100
threshold ← bg_max + tol
bg_mask ← volume ≤ threshold             # single-sided:
everything at/below threshold = background
return bg_values, bg_max, threshold, bg_mask
```

Mode 1 — `remove_outside_brain` (`_ExtRm` suffix): zero background pixels *outside* the brain only; everything inside the brain mask is kept untouched regardless of intensity.

```
bg_mask ← _threshold(...)
to_zero ← (~brain_mask) AND bg_mask
result ← volume.copy(); result[to_zero] ← 0
```

(Note: as actually wired in `_widget.py`*'s* `_on_run`*, Mode 1's saved* `brain_only` *is* `volume × eroded_mask` *directly — the entire exterior zeroed, not* `remove_outside_brain`*'s more selective "background-only" result — see §9.)*

Mode 2 — `remove_global` (`_NoBG` suffix): zero every background-threshold pixel in the *whole* stack, inside the brain included — this is the input the Pixel Classifier route needs (background must be zero everywhere for its own `volume > 0` → smooth → 3D connected-components pipeline to find clean isolated blobs).

```
bg_mask ← _threshold(...)
if signal_erosion_voxels > 0:
    signal_mask ← erode_signal_2d(~bg_mask,
signal_erosion_voxels)   # shrink what SURVIVES
    bg_mask ←
~signal_mask                                            #
thresholding, not the mask
result ← volume.copy(); result[bg_mask] ← 0
```

`erode_signal_2d(mask, iterations)` erodes one Z-slice at a time (`binary_erosion` per 2D slice), not a single 3D erosion — a plain 3D erosion also eats `iterations` slices off the top *and* bottom of a blob's own Z-extent, which for a thin blob can delete it outright; per-slice erosion only ever shrinks each slice's in-plane contour.

Mode 3 — `fill_outside_brain_random` (`_RndFill` suffix, presentation-only, not fed into any labeling route):

```
bg_mode ← _estimate_background(...)
tol ← 0.001 × (max(volume) - min(volume))          # ±0.10% fill
window around the mode
result ← volume.copy()
result[outside brain] ← uniform_random(bg_mode - tol, bg_mode +
tol), one value per outside voxel
```

`_widget.py` — Tab 1 handlers

`_on_open()` — background-thread wrapper around `_io.load_file(path)`; on completion, adds every returned channel as its own `Image` layer (`_add_channels`) and reports channel count + shape.

`_on_load_labels()` — loads a saved `.tif` directly as a `Labels` layer: reads with `tifffile.imread(...).astype(int32)`, scales it to match whatever stack is already open (`self._state["metadata"]["scale"]`, else (1,1,1) with a warning), and names it `<active Image layer>_labels` (the file's own stem if no Image layer is open), replacing a same-named layer — the same convention the Pixel Classifier's Create Labels uses.

`_on_run()` — the main button:

```
validate: model file exists, an Image layer is selected and 3D
read Threshold/Erosion/Background-mode/BG-Threshold/Signal-Erosion
from their sliders
device ← cuda, else mps, else cpu
background thread, wrapped in capture_live_output (surfaces
MONAI's tqdm bar +
    _background.py's own print() lines into the GUI's log view,
not just a terminal):
    brain_mask, brain_only, eroded_mask ← run_inference(volume,
model_path, threshold,
                                                        device,
erosion_voxels)
    if bg_mode == 1 (_ExtRm):  brain_only ← volume × eroded_mask
    if bg_mode == 2 (_NoBG):   brain_only ← remove_global(volume,
```

```
eroded_mask, ...) × eroded_mask
    if bg_mode == 3 (_RndFill): brain_only ← fill_outside_brain_random(volume, eroded_mask)
    # bg_mode == 0 (Off): brain_only stays exactly run_inference()'s own (volume × eroded_mask)
on completion: add <stem>_brain_mask (Labels, cyan, opacity 0.4) and
    <stem>_brain_only<suffix> (Image) layers (suffix per _BG_SUFFIX), replacing stale ones;
    save the ticked outputs (zlib); if bg_mode != 0 ALSO always save the raw input
    as <stem>_original.tif (the raw signal skin protection needs — every brain_only
    variant is masked); end the job (email if it ran past the threshold)
```

`<stem>` is the processed layer's own name; the folder is `_output_dir()` (§9).

---

4. Stage 2 — Label creation

UI: Tab 2 — Create MG Labels. Two independent routes, each shown whenever an Image layer with its background-mode suffix exists anywhere in the viewer (`_NoBG` → Pixel Classifier, `_ExtRm` → Cellpose-SAM; both can show at once): **Pixel Classifier** (`_labeling.py`'s `create_labels`, a plain 3D connected-components engine) and **Cellpose-SAM** (`_cellpose_seg.py`'s `run_full_pipeline`, a trained-network `do_3D` segmentation + several correction stages). The Cellpose-SAM route can chain the Stage 3 auto-correction (§5) and then a **Sanding** pass (`_sanding.py`); the Pixel Classifier route chains nothing. **Goal**: turn `brain_only` into a `Labels` layer, one integer ID per cell.

`_labeling.py` — Pixel Classifier: true 3D connected components

The engine is a plain 3D connected-components labeling (the UI group is titled "Pixel Classifier — 3D Connected-Component Labels") — `create_labels()` dispatches to whichever of 3 backends is fastest (detected once at import time: **CUDA** via CuPy/cupyx > **Apple MPS** → falls back to threaded CPU, since MPS itself has no `ndimage` ops > **CPU**,

`scipy.ndimage` + a `ThreadPoolExecutor` at 50% of cores). All three backends implement the identical 6-step algorithm; only the array library and slice-loop parallelism strategy differ.

`create_labels(volume, sigma_xy, sigma_z, min_volume, min_hole_size, final_min_fraction)`:

```
1. binary ← volume > 0
2. blurred ← gaussian_filter(binary, sigma=(sigma_z, sigma_xy,
sigma_xy))
   smooth_mask ← blurred > 0.5          # re-thresholded smooth
binary mask —
                                          # this is what actually
connects/
                                          # disconnects nearby
blobs, not the
                                          # raw thresholded signal
3. for each Z slice:
       fill any enclosed background hole whose area <
min_hole_size
       (min_hole_size <= 0 → fill every hole unconditionally, the
        original behaviour)
4. labeled, n_objects ← connected_components(smooth_mask,
structure=ones(3,3,3))
                          # 26-connectivity: any of the 26
neighbors counts
5. threshold ← max(1, round(final_min_fraction × min_volume))
                          # NOT min_volume itself — same golden-
ratio
                          # safety-net relaxation
_cellpose_seg.py's
                          # final_min_size_cleanup() uses, since
this route
                          # has no later merge/reattach stage to
give a
                          # gray-zone object a second chance
   remove every blob smaller than threshold
6. renumber surviving blobs 1..N by DESCENDING volume (label 1 =
largest)
return int32 labels, 0=background
```

The CUDA path (`_create_labels_cuda`) runs steps 1–2 and the volume/threshold math as vectorized CuPy array ops (`cp.bincount` for per-label voxel counts instead of a Python loop), loops step 3 per Z slice on the GPU, and frees the CuPy + PyTorch memory pools (`_free_gpu_cache()`)

before returning — including on the exception path, so a mid-run CUDA OOM still falls back to CPU cleanly (`create_labels()`'s own `try/except` around the CUDA call). The threaded-CPU path (`_create_labels_threaded`, also used for Apple MPS) runs step 3's per-slice hole fill via a `ThreadPoolExecutor`, everything else as a single `scipy.ndimage`/`numpy` call over the whole volume.

`min_hole_size` mechanism (`_fill_holes_capped_gpu` / the CPU per-slice closure): unconditional `binary_fill_holes` fills every enclosed background region no matter its size — much like Cellpose's own default hole-filling (see §4's Cellpose-SAM section), a single-voxel prediction artifact and a genuine internal structural void look identical to it. A positive `min_hole_size` switches to an area-limited fill (`skimage.morphology.remove_small_holes` on CPU; a manual `label()` + `bincount()`-based size filter on GPU, since cupyx has no equivalent), leaving any hole at or above the floor standing as real background.

`_cellpose_seg.py` — Cellpose-SAM: `do_3D` + 4-stage cleanup pipeline

Same math as the project's earlier standalone `krendl_do3d.py` CLI script, refactored into importable functions. No GT-based relabeling/scoring here (that stays a CLI/research workflow, see §8) — this produces a clean, sequentially-labeled instance mask ready for manual correction.

`predict_flows(volume, model_path, anisotropy, gpu)` — the one genuinely GPU-bound, expensive step: a single Cellpose-SAM network forward pass producing a per-voxel flow field (`dP`) and cell-probability map (`cellprob`), with neither `cellprob_threshold` nor `flow_threshold` applied yet — both are cheap, CPU/light-GPU post-processing steps applied afterward. Split out on its own (mirroring `_inference.py`'s `predict_probability`/`postprocess_probability` split for MONAI) so a cellprob sweep can call this once and cheaply re-threshold many candidate values without repeating the expensive network pass:

```
model ← CellposeModel(pretrained_model=model_path, gpu=gpu)
_, flows, _ ← model.eval(volume, do_3D=True,
anisotropy=anisotropy, z_axis=0,
                         channel_axis=None, diameter=None,
normalize=True,
                         augment=False, compute_masks=False)  #
compute_masks=False
                                                              #
skips mask formation
```

```
dP, cellprob ← flows[1], flows[2]
return model, dP, cellprob, volume.shape
```

`save_flow_cache` / `load_flow_cache` — persist `predict_flows()`'s expensive output to disk (`np.savez_compressed`) plus a fingerprint (model checkpoint path, anisotropy, volume shape/dtype). Exists because mask formation (`masks_from_flows`'s own internal `follow_flows()` re-run) needs its own separate chunk of GPU memory and can CUDA-OOM *even after* a multi-hour network pass already succeeded — without a cache, that OOM silently destroys the entire network pass (Python's own except-block variable cleanup, confirmed by reading `_widget.py`'s own handler, keeps only `str(exc)`, nothing else). `run_full_pipeline()`'s `flow_cache_path` saves the cache right before mask formation starts and deletes it automatically the moment mask formation succeeds — it only survives on disk if that specific step crashes, and a stale/mismatched cache (different fish/model/anisotropy) is detected via the fingerprint and ignored, never silently reused.

`masks_from_flows(model, dP, cellprob, shape, cellprob_threshold, flow_threshold, min_size, max_size_fraction, niter, min_hole_size, progress_cb)` — cheap: forms instance masks from an already-computed flow field. `do_3D=True` is baked in. `flow_threshold` is accepted only to match `do_3D`'s call signature — Cellpose's own `compute_masks()` only applies its flow-error QC filter when `do_3D=False` (confirmed directly in `cellpose/dynamics.py`; a call-count spy test found 0 calls under `do_3D=True`), so it's a documented no-op here. `niter=None` is resolved to Cellpose's own default of 200 (calling `_compute_masks()` directly, as this function does, skips `eval()`'s own resolution of that default, which would otherwise crash with a `TypeError` on `range(None)`).

```
if niter is None: niter ← 200
report progress (no further progress prints until this whole step
finishes —
                  neither this function nor Cellpose's own
follow_flows()/
                  compute_masks() log anything while it runs)
monkey-patch cellpose.utils.fill_holes_and_remove_small_masks →
    _make_capped_fill_holes(min_hole_size)   # see below
try:
    masks ← model._compute_masks(shape, dP, cellprob,
flow_threshold, cellprob_threshold,
                                 min_size, max_size_fraction,
niter, do_3D=True)
```

```
finally:
    restore the original fill_holes_and_remove_small_masks
return masks as int32
```

`_make_capped_fill_holes(min_hole_size)` — builds a drop-in, size-aware replacement for Cellpose's own hole-filling (which unconditionally calls `fill_voids.fill()` over each predicted mask's full 3D crop, no size threshold at all — the same category of bug `create_labels()`'s own `min_hole_size` was built to fix). `min_hole_size<=0` keeps Cellpose's exact original unconditional behavior. A positive value switches to a **per-Z-slice** `skimage.morphology.remove_small_holes` loop, not one 3D call over the whole crop — a real per-slice-small hole that persists across many Z-slices is one 3D-connected void whose *total* volume can dwarf the threshold, so a plain 3D `remove_small_holes()` judges it "too big to be noise" and leaves a visible ring on every slice it spans; looping per-slice avoids that (mirrors `_labeling.py`'s own per-slice treatment).

`gmm_cleanup(masks)` — 3-component Gaussian Mixture Model on `log1p(volume)` of every raw predicted object, separating a noise / gray-zone / real-cell population and dropping everything below the auto-detected gray→cell intersection point:

```
if fewer than 3 objects: return unchanged (not enough data to fit
3 components)
x ← log1p(each object's voxel volume)
gmm ← GaussianMixture(n_components=3).fit(x)
sort the 3 fitted components by mean: small, mid, large
cutoff ← the point where the mid and large components' fitted
Gaussian
          curves cross (closed-form solution of two Gaussians'
log-density
          equality; falls back to the midpoint of the two means if
the
          quadratic has no real root in range)
remove every object smaller than cutoff
return masks, cutoff, n_removed
```

`krendl_safe_merge(masks, max_gap, min_contact, gt_min, scale_zyx)` — merges only genuinely small fragments (< `gt_min`, the smallest true GT-labeled cell volume ever confirmed) into their nearest larger neighbor, when either close enough or touching with enough contact area:

```
repeat up to 200 passes:
    candidates ← every fragment below gt_min, smallest first
    for each candidate fragment f:
        find the nearest larger-volume object t (bbox-proximity-
prefiltered
        in voxel space, using max_gap converted to a voxel margin
via the
        FINEST axis — safe over-inclusion, never wrongly rejects a
close pair)
        gap ← min distance from f's mask to t's mask, via a
physically-scaled
              EDT (distance_transform_edt(~f_mask,
sampling=scale_zyx)) —
              NOT plain voxel-index distance: this project's
voxels are
              anisotropic (Z=1.0µm, XY=0.174µm typical), so an
unscaled EDT
              would treat a "2 voxel" gap as 2.0µm along Z but
only 0.35µm
              in-plane, a ~5.7x orientation-dependent
inconsistency
        do_merge ← gap <= max_gap (µm)
        if not do_merge: fall back to contact-area check
            (dilate f by 1 voxel, count overlap with t) >=
min_contact
        if do_merge: absorb f into t, update t's cached vol/
centroid/bbox
    if nothing merged this pass: stop
return masks, total_merges
```

`min_contact`'s fallback branch is largely dead code in practice at any reasonable `max_gap`: the smallest possible EDT gap between two literally-touching-but-disjoint fragments is exactly one voxel spacing, always ≤ a positive `max_gap` — confirmed both by this structural argument and empirically against real production GT data (see the project's session history).

`large_contact_merge(masks, large_contact)` — a separate, later, size-agnostic merge: any two objects (regardless of size) sharing a contact area ≥ `large_contact` voxels get merged, catching blobs split through a thick junction rather than a thin neck. Same repeat-until-stable structure as safe-merge, contact measured the same way (1-voxel dilation + overlap count); the larger of the two objects' ID always survives.

`final_min_size_cleanup(masks, gt_min, fraction=0.618)` — the last-resort safety net, run after GMM/safe-merge/large-contact: removes anything still below `fraction × gt_min` (golden ratio by default — strict enough to reject debris, lenient enough not to reject a legitimately smaller-than-average real cell). Nothing upstream is guaranteed to remove every possible debris object (GMM judges by population statistics; safe-merge/large-contact only act when a mergeable neighbor exists), so this stage is a floor under all of them, not a replacement.

`relabel_sequential(masks)` — renumbers surviving labels 1..N with no gaps.

`run_full_pipeline(volume, model_path, cellprob, flow, anisotropy, max_gap, min_contact, large_contact, gt_min, ..., flow_cache_path)` — glues every step above together:

```
1. dP, cellprob_map, shape ← predict_flows(...) or reuse
precomputed_flows /
   a matching on-disk flow_cache_path (see save/load_flow_cache
above)
2. masks ← masks_from_flows(...)                                  #
raw prediction
   delete flow_cache_path once this succeeds
   raw_masks ← masks.copy()  # kept in stats['raw_masks'] — this
is what
                              # _widget.py always auto-saves as
<stem>_cp.tif
3. masks, gmm_cutoff, gmm_removed ← gmm_cleanup(masks)
4. masks, safe_merges ← krendl_safe_merge(masks, max_gap,
min_contact, gt_min, scale_zyx)
5. masks, lc_merges ← large_contact_merge(masks, large_contact)
6. masks, final_removed ← final_min_size_cleanup(masks, gt_min,
final_min_fraction)
7. masks, n_final ← relabel_sequential(masks)
                           # the returned masks are what _widget.py
always
                           # auto-saves as <stem>_cp_krendl.tif
return masks, stats{n_raw, n_after_gmm, n_after_safe_merge,
n_after_large_contact,
                    n_after_final_min_size, n_final,
gmm_cutoff_vox, gmm_removed,
                    safe_merges, large_contact_merges,
```

`final_min_threshold_vox,`
`final_min_removed, raw_masks}`

`rerun_single_cell(volume, labels, label_id, model_path, ..., pad_z=15, pad_xy=40)` — "fix one mis-segmented cell" without a full-fish re-run: crops to that label's own padded bounding box, re-runs the *entire* `run_full_pipeline()` on just the crop (safe to reuse unmodified — GMM cleanup already no-ops below 3 objects, and the merge/cleanup stages use a fixed `gt_min` floor, not crop-local population statistics), then zeroes the old label everywhere and splices back only the crop-result objects that actually overlap the original label's own footprint — anything else the crop's own `do_3D` pass happens to also detect (e.g. a neighboring cell caught by the padding) is discarded, never duplicated into the full volume. If the crop genuinely reveals more than one real cell where there was one label before, all of them are kept as brand-new label IDs appended after the current max. The splice is foreign-protected: a kept object is written only into voxels that are background once the old label is cleared, so a neighbouring label or skin (-1) in the crop is never overwritten; the voxels left to a neighbour are counted in `info["n_protected_vox"]` and reported in the status line, and a piece with nothing left to write gets no ID.

`_sanding.py` — cavity fill + sigma-softening pass, chained after any label-regenerating tool

Runs *after* a label's shape has already been rebuilt (auto-correction, Correct Label, etc.). Two per-label steps, both foreign-protected (a neighbor's already-claimed voxels are never taken): first (optional, on by default) **`fill_label_holes()`** fills every 3D-enclosed cavity inside the label — a real vesicle the intensity threshold excluded as "not signal" — then **`sand_label()`** Gaussian-blurs the label's own binary mask and re-thresholds at 0.5, rounding off small jagged/blocky voxel-scale steps without meaningfully reshaping the cell. Purely geometric, no image/intensity involved.

`sanding_pad(sigma_xy, sigma_z)`: `max(10, round(3 × max(sigma_xy, sigma_z)) + 3)` — enough bbox padding for the blur to round a boundary without being clipped by the crop edge. **`fill_holes_pad()`**: a fixed 10 voxels (no blur, so less room needed).

`sand_labels_stack(labels, sigma_xy=0.7, sigma_z=0.7, pad=None, min_volume, final_min_fraction=0.618, skin_label_id, fill_holes=True, progress_cb)`:

```
for each positive label ID in the volume (skin -1 is never filled
or sanded):
    if fill_holes:
        new_labels, hole_info ← fill_label_holes(new_labels, lid,
pad=fill_holes_pad())
        record voxels filled + the Z slices they landed on
    new_labels, info ← sand_label(new_labels, lid, sigma_xy,
sigma_z, pad)
                        # per-label Gaussian blur + re-threshold +
foreign-
                        # label exclusion (see
_labeling.sand_label, §5)
    record applied/skipped (+ reason) per label
if min_volume given:
    threshold ← round(final_min_fraction × min_volume)   # same
golden-ratio
                                                           #
safety net as
                                                           #
every other
                                                           #
final stage
    remove_debris(new_labels, threshold, skin_label_id)   # skin's
own small
                                                           #
fragments swept
                                                           # too,
if present
return new_labels, report{sigma_xy, sigma_z, n_cells_total,
n_cells_sanded,
                          n_cells_holes_filled,
n_cavity_voxels_filled,
                          holes_filled_detail, skipped_cells,
                          n_debris_fragments_removed}
```

Deliberately kept as its own small default (0.7/0.7 voxels), much smaller than the Pixel Classifier's own Smooth σXY/σZ (2.6/1.0 by default) — that pair decides whether raw blobs *merge* into one 3D instance before labels even exist; sanding runs after labels already exist and is foreign-protected (structurally can't merge two cells), so its only job is polishing edges, not reshaping.

`_widget.py` — Tab 2 handlers

Route auto-detection (`_update_labels_section_visibility()`): the Pixel Classifier and Cellpose-SAM sections show or hide based on whether a `_NoBG` or `_ExtRm`-suffixed Image layer exists anywhere in the viewer, not just whichever layer is currently active — both can be shown together (e.g. comparing routes on the same fish). The Edit MG Labels tools (Tab 3) are shown whenever any Labels layer exists.

`_on_create_labels()` (Pixel Classifier): background-thread wrapper running on `_active_layer()` (the selected Image layer, else the topmost), reading Smooth σXY/σZ, the Min hole size slider, Final min-size fraction and the config-held Min volume (`_current_min_volume()`), calling `create_labels()` (3D Gaussian smooth → re-threshold → per-slice hole-fill → true 3D connected components → volume filter → renumber — no per-slice overlap-linking step), adding the result as a new `Labels` layer named `<image>_labels` (not saved to disk).

`_on_run_cellpose_seg()` (Cellpose-SAM Segmentation): reads its input from the section's own **Signal layer** combo (`_cp_signal_combo`, pre-selecting the first `_ExtRm` layer) — never the active layer; validates the checkpoint file (`_is_valid_cellpose_checkpoint()` — checks it's a zip archive, the format `torch.save()` uses, to catch a wrongly-selected non-checkpoint file before a multi-hour run rather than deep inside `torch.load()`); when "Auto-correct" is ticked, also requires the Brain mask combo's layer (same shape) and the raw channel layer (`_raw_channel_layer()`: the signal layer's name with its `_BG_SUFFIX` and `_brain_only` stripped, open in the viewer, same shape) — all before segmentation starts. Then runs `run_full_pipeline()` in a background thread wrapped in `capture_live_output` (surfaces Cellpose's own otherwise-silent flow-following/hole-fill/small-object-removal step into the GUI log), with `flow_cache_path = <output>/<stem>_cp_flow_cache.npz`. On completion: adds the result as `<stem>_cellpose_labels`, **unconditionally** (no checkbox) writes `<stem>_cp.tif` (`raw_masks`) and `<stem>_cp_krendl.tif` (the final result) alongside the fish's other outputs, and — if "Auto-correct labels via contrast sweep after segmentation" is ticked — chains `_run_auto_correction_stage()` (the Stage 3 auto-correction pipeline, §5, run on the **raw channel** layer's data) as a further background stage, then `_run_sanding_stage()` if sanding is on.

`_on_rerun_single_cell()`: resolves the correct source Image layer and target Labels layer via `_resolve_rerun_layers()` (selected Labels layer wins if any name; else `<image>_cellpose_labels` → `<image>_labels` → a single same-shape Labels layer; ambiguity is reported, never guessed

— fixes a real earlier bug where a labels layer not literally named `<stem>_cellpose_labels` was rejected), then calls `rerun_single_cell()` in a background thread and splices the result back into the live layer.

Sanding UI: one shared checkbox (Tab 2 Common Settings, plus the Fill-interior-cavities checkbox there) and one Sigma XY/Z pair (Tab 3's Sanding group) drive sanding at every call site that regenerates a label's shape (Correct Label 2D/3D, Correct Adjacent Labels, and — chained as its own stage — both auto-correct entry points), not a separate control per tool.

---

5. Stage 3 — Label editing & correction

UI: Tab 3 — Edit MG Labels, working against a shared explicit Signal/Labels/Brain-mask layer selection (`_edit_labels_layer()`, see §1 "Layer-selection philosophy") plus a Label A / Label B selector pair most tools reuse. **Modules**: `_labeling.py` (the editing primitives), `_grow_correct.py` (auto-grow/until-stable orchestration), `_contrast_sweep.py` (calibrating the intensity threshold these tools use), `_auto_correction.py` (the full unattended per-cell pipeline chained after Cellpose-SAM Segmentation). This is the largest, most-iterated part of the plugin.

Shared building blocks

Every tool in this section that regenerates a shape from intensity shares one convention: **`candidate = image >= lo`** — a one-sided threshold, not a band. `hi` (the signal layer's own display ceiling) is shown for context only, never used to restrict the mask — a narrow contrast window like `[100, 101]`, chosen to make the display saturate into a clean silhouette, would otherwise exclude the cell's true bright interior (well above `hi`, just shown as saturated white) if `hi` bounded the mask.

Every tool also shares **foreign-label protection**: any pixel already claimed by a *different* existing label is excluded from the candidate mask before anything (connected components, watershed) runs — a foreign label can never be grown into, merged into, or even appear in a corrected shape, regardless of how the threshold or connectivity falls.

`_labeling.py` — editing primitives

`resort_labels(labels, sort_by, reverse)`: renumber every label 1..N by a chosen key — `"size"` (voxel count via `bincount`, natural = descending, largest→1), `"centroid_z/y/x"` (via `label_centroids_zyx`, natural = ascending), or `"complexity"` (skeleton branch count via `_statistics._skeleton_stats`, natural = descending, computed per-label in a thread pool). Only positive labels are ever touched by the final LUT remap — a sentinel like skin's `-1` would otherwise corrupt via numpy's negative-index wraparound (`lut[-1]` = last element).

`label_centroids_zyx(labels, label_ids)`: bit-identical to `scipy.ndimage.center_of_mass`, but computed from each label's own bounding box (`find_objects` + a per-label crop) instead of one whole-volume call — ~10-100x faster when a large negative sentinel (skin) is present, since scipy's own implementation takes a much slower path for negative-containing arrays.

`remove_debris(labels, threshold, skin_label_id=None)`: zeros every *spatially connected fragment* smaller than `threshold` voxels — evaluated per connected component (26-connectivity), not per raw label ID, so several small disconnected fragments still sharing one old ID (e.g. after a manual edit) are each judged on their own size, not their summed total. Does **not** renumber survivors. `skin_label_id`, when given and present, is swept too (skin's own bounding box computed manually since `find_objects()` never indexes negative values). The per-label sweep (`_remove_debris_from_crop`) clears every small fragment of a label in one `bincount`+boolean-index pass, not one array scan per fragment — a real historical bug (per-fragment `crop[cc==id]=0` scanning the whole crop each time) made this "hang" on a fragmented skin label with thousands of small pieces.

`split_label(labels, target_label, n_splits, sigma, min_distance, mode, z, image)` — watershed split into N parts:

```
mode="3d": surface = the mask's own EDT (splits at the
geometrically
           narrowest neck, regardless of what the underlying
signal looks like)
mode="2d": surface = raw signal intensity on slice z (seeds at
brightness
           peaks; cut follows the dimmest ridge — for when two
things only
           merge on ONE cross-section, where the 3D mask has no
real neck
```

```
           but the signal already dips)
1. crop to bbox(+pad)
2. Gaussian-smooth the surface (GPU if available)
3. h_maxima(surface, h) — TOPOLOGICAL prominence, not
peak_local_max's
   Euclidean-radius search: finds peaks separated by the thinnest
saddle
   regardless of how physically close their centers are. Auto-
reduces h
   from 50% of max down to 0.5% until >= n_splits peaks are found.
4. within each h_maxima region, seed = its own EDT/intensity
maximum;
   drop seeds closer than min_distance to an already-chosen one
5. watershed(-surface, markers=seeds, mask=mask_crop)
6. clear the 1-voxel interface where two parts directly touch
   (outer surface of each part untouched)
```

Part 1 keeps `target_label`; parts 2..N get fresh IDs above the current max.

`join_labels(labels, label_a, label_b)`: the inverse — every `label_b` voxel becomes `label_a`. One vectorized boolean assignment, no crop/GPU needed.

`correct_label_from_intensity(labels, image, label_id, z, lo, hi, pad)` — Correct Label's 2D engine: within `label_id`'s own padded bbox on slice `z`, `candidate = image >= lo`, foreign-excluded, painted in wholesale (not connectivity-restricted to the label's existing footprint — a human reading the same contrast window would count a visible blob as part of the cell whether or not it happens to already touch). Raises rather than silently emptying the label if nothing survives.

`correct_label_from_intensity_3d(labels, image, label_id, lo, pad, ..., auto_grow, growth_step, max_iterations, until_stable, max_stability_passes, resolve_adjacent, z_extent_pad)` — Correct Label's 3D engine, the most complex function in the plugin. NOT a bounding box over the whole cell — loops one Z-slice at a time, each with its own locally re-derived neighborhood:

```
zmid ← z_orig_min + ceil((z_orig_max - z_orig_min) / 2)
walk z from zmid down to z_orig_min, then from zmid+1 up to
z_orig_max:
    _or_seed(z):  # before correcting z, OR-union z-1 and z+1's
footprints
                  # (within the label's fixed original range) into
```

```
z's own
                  # current footprint as watershed seed context —
an
                  # already-walked neighbor contributes its JUST-
corrected
                  # shape, a not-yet-walked one its ORIGINAL
shape. OR, not
                  # AND (the source pseudocode's own literal "AND"
was a
                  # stated user error — AND would erase real
signal any
                  # time a neighbor's shape isn't already
identical).
                  # Pasted only into empty background or skin
(-1), never
                  # a different real label's territory.
    _correct_slice_full(z):
        if another label (a real cell, or skin -1) is present in
z's own local area
        and resolve_adjacent is True:
            jointly re-derive via the same marker-seeded watershed
            Correct Adjacent Labels uses
(_correct_label_group_2d_core,
            focus_ids=[label_id]) — only label_id's OWN resulting
side
            is ever written back
        else:
            plain single-label 2D correction
(_intensity_correct_2d)
        if auto_grow and result touches its own local window's
edge:
            retry this ONE slice at pad += growth_step, up to
max_iterations
        if until_stable:
            keep re-running THIS SLICE (reseeded from its own last
result)
            until it matches the previous attempt, or
max_stability_passes
extend outward past the original [zmin, zmax] one slice at a time
(up to
    z_extent_pad, a FIXED value never inflated by growth): copy
the
    current footprint forward (foreign-protected), then run the
same
```

```
    local correction there; undo and stop the moment nothing
supports it
    — the true edge has been found
remove_debris_for_label() cleans up any small disconnected
leftover
```

Every slice's own correction is *locally anchored* — re-derived fresh from wherever the label actually sits one slice ago, so it can only ever reach `pad` beyond its own verified position, never balloon into unrelated distant signal the way one whole-cell-sized window would. `resolve_adjacent=False` treats every other label as purely excluded territory instead of jointly resolving against it — used for skin's own sentinel, which borders essentially every real cell and shouldn't be watershed-split against each one.

No missing-slice repair pass here, unlike `correct_label_2d_stack()` below: `_or_seed()` guarantees any slice within `[z_orig_min, z_orig_max]` starts from non-empty content whenever either Z-neighbor has real content, and `_correct_one_slice()`'s snapshot/restore means even a failed attempt only ever reverts to that already-non-empty state, never to zero — confirmed by reading both mechanisms together, not assumed. Only this fully-independent, no-cross-slice-seeding 2D engine can leave a slice genuinely empty after a single flaky attempt.

`fill_label_holes(labels, label_id, pad=10)` — the cavity fill behind §4's sanding pass: `binary_fill_holes` on the label's own 3D mask inside its padded bbox; only voxels that are currently background (0) become the label, so a different label sitting inside an apparent cavity is never absorbed. Returns which absolute Z slices received filled voxels.

`sand_label(labels, label_id, sigma_xy, sigma_z, pad)` — the per-label engine behind §4's sanding pass: 3D Gaussian-blur the label's own binary mask, re-threshold at 0.5, foreign-excluded; skips (reports why) if the result collapses to nothing or loses contact with the original footprint.

`_intensity_correct_2d` / **`_intensity_grow_2d`** — the shared single-slice threshold core every 2D tool above calls into: crop to `seed_mask`'s bbox+pad, `candidate = image >= lo` minus foreign territory, fill the *whole* crop (not connectivity-restricted to the seed).

`correct_adjacent_labels_2d(labels, image, label_a, label_b, z, lo, pad, sigma)` — corrects two touching labels on one slice *simultaneously*, boundary placed by a **marker-seeded** watershed (`markers = each label's own existing footprint`, not auto-detected peaks — Split

Label's blind peak-finding was tried and rejected here, since a combined region can have more than one real intensity dip and lock onto the wrong one). The working rectangle is sized from `label_a` alone (+pad), not the union — `label_b` only participates via whatever falls inside that rectangle.

`correct_label_group_2d` / **`_correct_label_group_2d_core`** — the N-label generalization of the above, used by `grow_correct_label_2d()` (Correct Label 2D with auto-grow, Correct Adjacent Labels with auto-grow or until-stable) and (crop-only, no full-volume copy) by `correct_label_from_intensity_3d`'s per-slice joint branch. `focus_ids` optionally scopes the working rectangle to a subset of the group (mirrors `label_a`-only scoping); markers are always each label's own existing footprint, watershed runs unmasked over the whole crop then intersects with the real candidate signal so a diagonally-touching island still gets an owner.

`touching_groups_for_stack` (general-purpose utility, not on the current pipeline's own critical path — replaced there by the per-cell engine's own adjacency resolution): per-Z-slice union-find of mutually-touching label clusters (`_touching_pairs_on_slice` + `_union_find_groups`).

`copy_label_to_adjacent_slice(labels, label_id, z_src, direction)`: copies one label's 2D footprint onto the next/previous slice, clearing its own old footprint there first, foreign-excluded (reports how many pixels were dropped to foreign territory rather than pretending an exact copy).

Skin protection: `skin_voxel_count(labels, skin_label_id=-1)` — the single, array-content-only definition of "is skin already protected" every path checks (never widget state). `seed_skin_label(labels, brain_mask, skin_label_id=-1)` — bulk-fills every background voxel outside the brain mask with the sentinel `-1` (never overwrites a real label, never touches inside the brain); this is only a coarse seed. `trim_skin_label(...)` — a thin wrapper around `correct_label_2d_stack` (below) trimming that bulk seed down to real signal-supported territory, per-slice only (skin has no cross-slice-continuity problem to solve and spans nearly the whole Z range, so the 3D walk's machinery would be pure waste — the same reason 3D correction of an *existing* skin label is structurally blocked in the interactive UI). No brain-mask clamp — a legitimate inward correction along skin's real inner boundary is never blocked; a `remove_debris(..., skin_label_id=...)` pass run right after sweeps up

any small stray blob skin absorbed instead. `remove_label(labels, label_id)` — plain bulk clear, not connectivity-restricted, used to drop skin's sentinel when no longer needed.

`correct_label_2d_stack(labels, image, label_id, lo, pad, ..., n_workers, max_repair_attempts=2)` — corrects one label across its *whole* Z range, slice by slice, **independently and in parallel** (`ThreadPoolExecutor`, 75% of cores by default, `n_workers` overridable so a caller already parallel across cells — the auto-correct waves — doesn't multiply thread pools), each slice via `grow_correct_label_2d()` (the exact function the interactive Correct Label 2D button calls) on a one-slice *view*, not a full-volume copy per slice — keeping RAM bounded regardless of how many slice threads run concurrently. Wherever another real label is present, the boundary is jointly resolved via the same marker-seeded watershed, but only `label_id`'s own resulting side is ever painted back — guaranteed by watershed's own marker semantics (a label's own existing pixels are always its own marker, which watershed can never reassign), not a separately-enforced rule. After the main per-slice loop, `_repair_missing_label_slices()` runs: any slice where `label_id` existed in the function's own input but ends up completely absent from the result gets up to `max_repair_attempts` independent re-seed-from-original-footprint-and-retry passes (foreign-protected — never reclaims a pixel now legitimately owned by a different real label) before being reported as a genuine gap (`slices_repaired`/`slices_missing_after_repair` in the returned report) — a safety net this fully-independent-per-slice design needs and the 3D whole-cell engine below structurally doesn't (see its own note). `trim_skin_label()` is a thin wrapper calling this with `label_id = skin_label_id`, plus its own second, redundant repair layer keyed off the brain mask directly (`_repair_missing_skin_slices()`, merged into the same report fields) when a `brain_mask` is passed to it.

`_grow_correct.py` — auto-grow/until-stable orchestration

`grow_correct_label_2d(labels, image, label_ids, z, lo, initial_pad, growth_step, max_iterations, sigma, until_stable, max_stability_passes)` — wraps `correct_label_group_2d` in two nested loops:

```
group ← {label_ids}  (a single int for Correct Label; [A, B] for
Correct
                      Adjacent Labels — the rectangle stays
scoped to
```

```
                      the FIRST id even as the group grows)
for each stability pass (1 if until_stable=False, else up to
max_stability_passes):
    for attempt in 1..max_iterations:
        correct the current group at the current pad
        new_neighbors ← labels the ORIGINAL group's own footprint
(not a
                        folded-in neighbor's) GENUINELY touches,
that
                        aren't in the group yet
        if new_neighbors: group |= new_neighbors; redo at the SAME
pad
        elif the focus id(s) still touch their own rectangle's
edge:
            pad += growth_step; retry
        else: converged, break
    if until_stable and this pass's result on every group member
matches
       the immediately preceding pass's: stop (stable)
    else: reseed the next pass from this pass's own result
```

Neighbor discovery is deliberately narrow: only the *originally requested* label's own genuine touching-adjacency counts (never "merely present in the box," which would keep finding something as the box grows; never an already-folded-in neighbor's own further touches, which could cascade the group outward indefinitely).

`grow_correct_label_3d(labels, image, label_ids, lo, ...)` — a thin pass-through to `correct_label_from_intensity_3d(auto_grow=True, ...)`: all growth and stability happens *inside* that function's own per-slice walk, never as a whole-cell retry at a bigger global pad.

`format_grow_report(report, mode)`: renders either shape into the same human-readable style (which slices grew, which took multiple stability passes, whether it converged) used throughout the plugin's status/report boxes.

`_contrast_sweep.py` — calibrating Correct Label's own `lo`

Answers "which `lo` best *reproduces* what Cellpose-SAM already segmented" — self-referential calibration against the model's own output, not independent GT (a deliberately different target from every GT-sweep tool in §8).

`select_calibration_samples(labels, scale_zyx, n_cells, slices_per_cell, edge_margin_um)`: picks the `n_cells` most morphologically complex cells (skeleton branch count) whose centroid sits ≥ `edge_margin_um` from the volume's own edge (a proxy for "not already touching skin"), each axis's margin capped at 30% of that axis's own extent so an un-capped physical-µm margin doesn't collapse Z's own thin window to nothing. For each, draws `slices_per_cell` Z-slices spread across the middle 60% of its own Z-extent (avoiding thin top/bottom cross-sections).

`default_lo_candidates(labels, image, samples, pad, n_steps)`: auto-scales the candidate range from the 1st–99.5th percentile of intensity *around the actual samples* — each sample's own 2D bounding box on its slice, grown by `pad` px (the same window the sweep crops), never the whole slice — not a hardcoded guess — robust to a different channel/bit-depth/normalization.

`sweep_contrast_lower_value(labels, image, samples, lo_candidates, pad, refine=True)`:

```
for each candidate lo (coarse pass, then fine pass):
    for each (label_id, z) sample:
        corrected ← _intensity_correct_2d(..., lo) result (IoU=0
on error,
                     doesn't abort the whole sweep)
        iou[sample] ← IoU(corrected, the sample's EXISTING
footprint)
    mean_iou[lo] ← mean(iou over all samples)   # jointly, not
each sample's
                                                  # own
independent best —
                                                  # stops one
outlier sample
                                                  # from swinging
the result
fine pass (refine=True): every lo within one coarse step either
side of
    the coarse winner, spaced 1 apart on an integer image (the
finest
    spacing that changes image >= lo) or a tenth of the coarse
step
    otherwise, not already tried
best_lo ← argmax(mean_iou over coarse + fine candidates)
```

The fine pass exists because the mean-IoU peak is sharp (on a real fish, 0.92 at its best `lo` and 0.86 two intensity units away): a coarse grid alone makes `best_lo` depend on where its points happen to land. Any non-positive label (skin `-1`) is treated as plain background during this sweep — otherwise the correction-under-test would see skin as foreign territory and exclude it, making the calibrated `lo` depend on whether skin happened to already be protected.

`_auto_correction.py` — the full unattended per-cell pipeline

`auto_contrast_correct_stack(labels, image, scale_zyx, brain_mask, skin_image, ...)` — chained onto the end of Cellpose-SAM Segmentation (§4) when its "Auto-correct" checkbox is ticked, or run standalone via the **Auto-correct Labels** button against any Labels layer. Both widget call sites pass `pad` = Correction Settings' Bbox padding for every cell **and** as `skin_pad` (the function's own `skin_pad=25` default is never used by the UI), plus the Calibrate tool's Cells/Slices/Edge margin/Sweep steps and its Bbox padding as `sample_pad` — the sweep's own sample-crop padding (`None` falls back to `pad`), the same one Protect Skin as Label's sweep uses. Six steps:

```
1. best_lo ← sweep_contrast_lower_value(...) via
select_calibration_samples/
             default_lo_candidates, minus lo_adjustment ("Reduce
best_lo by")
             — or lo_override (Auto-sweep off: the signal layer's
current
             contrast low), which skips the sweep and is used as
given

2. if skin (-1) already present in `labels`: REUSE it exactly as-
is,
       never re-seed/re-trim (protects any prior manual
correction)
   else:
       seeded ← seed_skin_label(labels, brain_mask)
       labels_with_skin, skin_report ← trim_skin_label(seeded,
skin_image
           or image, -1, best_lo, pad=skin_pad, ...)
       # same best_lo real cells get -- skin's own joint-
resolution
       # against a touching cell (§ above) needs no separate
margin
```

```
3. remove_debris(labels_with_skin, threshold, skin_label_id=-1)
   # sweeps whatever small stray blob skin's own unclamped trim
absorbed

   GATE: skin_voxel_count(...) == 0 after steps 2-3 → raise. No
cell is
   EVER corrected without verifiably-present skin protection.

4. resort_labels(sort_by="centroid_z")
   touching_skin_ids ← _labels_touching_skin(...)   # GEOMETRIC
test on
       skin's FINAL territory (dilate each cell by skin_touch_px,
check
       overlap) — NOT read from trim_skin_label()'s own
foreign_nearby
       report, which lists every label ever transiently folded
into a
       growth attempt (found to wrongly flag 33/33 cells on a real
fish)
   waves, boxes ← _compute_correction_waves(...)   # partitions
every
       cell into provably non-conflicting groups by each cell's
own
       worst-case reach (pad + growth_step*max_iterations); same-
wave
       cells run concurrently (75%-of-cores ThreadPoolExecutor),
later
       waves see all earlier waves' already-corrected state

5. for each wave, for each cell (in parallel):
       if touching skin: correct_label_2d_stack(...,
label_id=cell)
                         # same per-slice mechanism
trim_skin_label()
                         # itself uses — a cell meeting skin has
to do
                         # so on skin's own 2D-only terms
       else:             grow_correct_label_3d(..., cell)
       merge only this cell's own expanded box back into the
shared array
       (wave members' boxes never overlap, so this is race-free)

6. remove_debris(new_labels, threshold, skin_label_id=-1)   #
```

```
final
       whole-layer safety net

   (report-only) _flag_possible_non_microglia(...): flags any cell
with
       >= non_microglia_fraction (default 40%) of its volume
outside the
       brain mask, OR of its own boundary SURFACE within
skin_touch_px of
       skin — macrophages or MONAI-missed skin that Cellpose
segmented as
       a cell. Nothing is removed or changed; report only.
```

`format_auto_correction_report()` renders one consolidated report: the calibration, skin protection outcome, wave/parallelism summary, per-cell detail (reusing `format_grow_report()` for 3D cells, a matching `_format_2d_vs_skin_report()` for skin-touching ones), and the non-microglia flag list.

Both widget call sites save that text: `_on_autocorrect_labels` to `<output folder>/<labels layer>_autocorrect_report.txt`, `_run_auto_correction_stage` to `<stem>_autocorrect_report.txt` next to `_cp_krendl_ac.tif` — both through `_write_autocorrect_report()` (date/labels/signal header, overwrites, never raises so a failed write can't abort applying the labels); the chained sanding stages append `format_sanding_report()` to the same file via `_append_sanding_report()`. Build GT-Correction Package picks the newest one up.

`_widget.py` — Tab 3 (Edit MG Labels) handlers

Shared layer selection: `_edit_labels_layer()` / `_edit_signal_combo` / `_edit_mask_combo` — one explicit combo triple the user sets once, replacing the old implicit "whatever's active" fallback (see §1).

`_on_resort_labels()`: reads sort-by/reverse from the UI, calls `resort_labels()` in a background thread, writes the result back in place (`layer.data[:] = result; layer.refresh()` — not `layer.data = result`, which was found to race with napari's own async thumbnail update and crash; see §9).

`_on_remove_debris()`: threshold ← `max(1, round(Final min-size fraction × Min volume))` (Min volume from config, `_current_min_volume()`), calls `remove_debris(labels, threshold, skin_label_id=-1)`.

Label A/B validation: both spinboxes have minimum -1; module-level `_edit_label_id_error(label_id)` returns an error string for 0 (background) or any ID below -1, else `None`. Split Label, Join Labels, Correct Label, Copy Label to Adjacent Slice and Correct Adjacent Labels call it on every label they read and stop with "ERROR: Label A/B: …" in their status line; the two "Use selected" buttons (`_on_use_selected_label_a/b`) use it to refuse a selected 0 or other negative ID.

`_on_split_label()`: reads Label A, N splits, σ, min distance, mode (2D/3D; 2D also needs the shared Signal layer and uses the current slice), calls `split_label()`, writes the result back in place.

`_on_join_labels()`: reads Label A/B (each with a "Use selected" button reading `layer.selected_label`), refuses A == B, calls `join_labels()`.

`_on_correct_label()`: reads the shared Signal layer's live `contrast_limits` for `lo`, Label A, Correction Settings, mode (2D uses the current slice; 3D disabled outright for label `-1`, since a 3D correction of skin would walk nearly the whole fish — see `correct_label_from_intensity_3d`'s own note). 2D: `grow_correct_label_2d()` if Auto-grow is on (no until-stable in 2D), else `correct_label_from_intensity()`. 3D: `grow_correct_label_3d()` if Auto-grow **or** Keep-until-stable is on, else `correct_label_from_intensity_3d()`. Afterward, if sanding is enabled (§4), fills cavities (if that checkbox is on) and sands every label of the resulting group.

`_on_correct_adjacent_labels()`: reads Label A/B (must differ) + current slice; `grow_correct_label_2d(label_ids=[A, B], ...)` if Auto-grow or Keep-until-stable is on, else `correct_adjacent_labels_2d()`; fills cavities and sands every label of the group afterward if enabled.

`_on_copy_label_to_adjacent_slice()`: reads label + direction, calls `copy_label_to_adjacent_slice()`.

`_on_protect_skin()`: refuses ("Skin is ALREADY protected…") if `skin_voxel_count() > 0` already. With Auto-sweep on, runs its own contrast sweep on the Signal layer — the same functions and sample parameters as "Calibrate Correct-Label Contrast" (`_lo_sweep_sample_params()`, including that tool's Bbox padding), not a reused result — and subtracts "Reduce best_lo by"; with Auto-sweep off (or if no samples can be found) it uses the Signal layer's current contrast low. Then `seed_skin_label` + `trim_skin_label` (Correction Settings, `brain_mask` passed for the missing-slice repair) on the skin image from `_resolve_skin_signal_image()` (the raw channel

behind an `_ExtRm`/`_NoBG`/`_RndFill` Signal layer, else the Signal layer itself), then `remove_debris(..., skin_label_id=-1)`. On completion sets the Signal layer's contrast to `(lo, _adaptive_contrast_high(...))` when the sweep ran, and fills the skin report box. 30-minute hang watchdog.

`_on_remove_skin_label()`: calls `remove_label(labels, -1)`.

`_on_autocorrect_labels()`: background-thread wrapper around `auto_contrast_correct_stack()` against the shared selector's Labels/Signal/Brain-mask layers (skin image from `_resolve_skin_signal_image()`) — the same pipeline Cellpose-SAM Segmentation's own "Auto-correct" checkbox chains into, exposed as its own standalone button so it can be re-run against any existing Labels layer, not just fresh segmentation output. On completion writes the labels back in place, sets the Signal layer's contrast, fills the skin report box, saves the report to `<output>/<layer>_autocorrect_report.txt`, and chains `_run_sanding_stage_ac()` (fill cavities + sand, report appended; never saves the labels) if sanding is on. Wrapped in the hang watchdog (`_hang_watch_start`/`stop`, 30-minute `faulthandler.dump_traceback_later()`) given how long a full-fish run can take.

"Hide skin label" toggle (`_on_toggle_skin_visibility()`): a purely visual `DirectLabelColormap` swap on the shared selector's Labels layer (never touches `layer.data`) making skin render transparent — a `_FastDirectLabelColormap` subclass bypasses napari's own slower numba-dict backend selection for this one colormap (a real ~2x measured fix for 3D-display slowness), verified to produce byte-identical output to the base class.

"Drift View in 3D" (`_on_toggle_drift()`, a Tab 6 General tool): Start/Stop + speed slider (1–100, default 20) driving a ~30 fps `QTimer`-ticked camera rotation (`viewer.camera.angles`, composed via a real `scipy.Rotation` each tick rather than incrementing 3 Euler angles independently — napari's own camera API restricts its middle axis to ±90°, which an increment-based approach visibly stalls against). Camera-only; never touches any layer's data.

---

6. Stage 4 — Statistics

UI: Tab 4 — Statistics, with a scrollable per-column output checklist (defaults matching core microglia morphology; several columns Nathalie's review flagged as marginal/unvalidated default OFF — see

§9). **Module**: `_statistics.py`, one function (`compute_stats`) producing a `pandas.DataFrame`, one row per label, 46 columns always and up to 51 with the optional intensity/region inputs (`_widget.py` adds `is_volume_outlier`, so the CSV has up to 52). **Goal**: turn a corrected Labels layer into a per-cell CSV.

Three-phase pipeline (`compute_stats`)

Phase 1 — batch regionprops, GPU or CPU (`_batch_regionprops`): one vectorized pass over every label at once — `cucim.skimage.measure.regionprops_table` on GPU (CuPy) if available, else `skimage.measure.regionprops_table` on CPU; both requesting the same property list (`label, area, centroid, bbox, inertia_tensor, inertia_tensor_eigvals, axis_major_length, axis_minor_length, solidity, extent`). From this table, vectorized post-processing derives:

```
IT ← the 3x3 inertia tensor per label, stacked
eigvals, eigvecs ← eigh(IT)                    # symmetric
eigensolve, all labels at once
principal_axis_dir[i] ← "Z"/"Y"/"X", whichever axis the LONGEST
eigenvector
                          (column 0) has its largest |component|
along
axis1_um ← axis_major_length × (that axis's own physical scale)
axis3_um ← axis_minor_length × mean(sz, sy, sx)
axis2_um ← (axis1_um + axis3_um) / 2            # not
independently measured
elongation ← axis1_um / axis3_um                # 1 = sphere, >1 =
elongated
bbox_dz/dy/dx_um ← bbox extent x physical scale
volume_um3 ← voxel count x (sz x sy x sx)
eq_diam_um ← (6 x volume_um3 / pi) ^ (1/3)       # equivalent-
sphere diameter
```

Phase 2a — per-label marching cubes + skeleton, threaded (`_slow_stats_worker`, one call per label in a `ThreadPoolExecutor` at 50% of cores): for each label's own bbox crop,

```
_surface_area(binary, scale_zyx):
    pad the crop by 1 voxel (avoids a boundary artifact)
    verts, faces ← marching_cubes(padded, level=0.5,
spacing=scale_zyx)
    return mesh_surface_area(verts, faces)        # μm², 0.0 on
any failure
```

```
_skeleton_stats(binary, scale_zyx):
    skeleton ← skeletonize(binary)                 # skimage,
topological thinning
    sk ← skan.Skeleton(skeleton, spacing=scale_zyx)
    bd ← skan.summarize(sk)                        # one row per
branch segment
    n_branches ← len(bd)
    n_endpoints ← count of branch-type == 1 (free ends, not
junction-junction)
    mean/max_branch_len_um ← mean/max of bd's euclidean-distance
column
    tortuosity ← mean(path_length / euclidean_distance) over
branches with
                 nonzero euclidean distance — 1.0 = perfectly
straight
    returns (0,0,0.0,0.0,1.0) on any failure (e.g. skan not
installed, empty skeleton)
```

Phase 2b — intensity stats, optional, also threaded (`_intensity_stats_worker`, only if an Image layer was given): per-label mean/sum/coefficient-of-variation of the raw signal intensity within the label's own mask.

Phase 3 — assembly + derived metrics + descriptions:

```
for each label:
    sphericity ← min(1.0, pi^(1/3) x (6 x volume)^(2/3) /
surface_area)
    surface_to_volume_ratio ← surface_area / volume
    branch_density ← n_branches / volume x 1e6          # per-µm³,
scaled for readability
    endpoint_density ← n_endpoints / volume x 1e6
    process_complexity ← n_endpoints x mean_branch_len / volume
    morphotype ← _classify_morphotype(sphericity, solidity,
elongation, n_branches, sav)
    description ← desc_fn(row)                           # rule-
based or an LLM backend
```

`_classify_morphotype(spher, solid, elong, n_br, sav)` — rule-based, priority order:

```
if elong > 3.5 and n_br <= 3:                          → "Rod-
shaped"
elif spher > 0.70 and solid > 0.80 and n_br <= 2:       →
"Amoeboid"        (activated)
```

```
elif n_br >= 6 and sav > 2.0 and spher < 0.55:          →
"Ramified"        (resting)
elif n_br >= 4 and spher < 0.65:                        →
"Intermediate-ramified"
else:                                                    →
"Intermediate"
```

Post-assembly, whole-dataset spatial statistics (`_spatial_stats`, `scipy.spatial.cKDTree` over all centroids at once):

```
tree ← cKDTree(centroids_um)
k=3 query → self + 1st NN + 2nd NN (k=2 fallback if only 2 cells
exist)
nearest_neighbor_dist_um, nearest_neighbor_2_dist_um ← those
distances
Clark-Evans 3D index:
    density ← N / bbox_volume(all centroids)
    expected_NND ← Gamma(4/3) x (3 / (4*pi*density))^(1/3)   #
Poisson-process expectation
    nearest_neighbor_ratio ← observed_NND / expected_NND      # <1
clustered, >1 dispersed
local_density_100um ← count of other centroids within a 100µm
radius (tree.query_ball_point)
depth_normalized ← centroid_z_um / max(centroid_z_um), clipped to
[0,1]
```

Post-assembly, optional brain-region assignment (`_assign_brain_regions` + `_polyline_side_and_dist`), only if a Shapes layer with `"line"`/`"path"` shapes and region names were given:

```
boundaries ← the drawn polylines, in µm, sorted by mean X of their
own vertices
             (ascending = most anterior first) -- the fish lies
along X,
             head at X=0; each boundary is meant to run top-to-
bottom (Y),
             dividing anterior (smaller X, left of the path) from
posterior
for each cell's centroid:
    for each boundary, nearest-to-first:
        find the closest point on ANY of the boundary's segments
(point-to-
        segment projection, not just to its two endpoints --
supports a
        curved multi-vertex polyline, not just a straight 2-point
```

```
line)
        side ← sign of the cross-product at that nearest segment
        if side says "posterior of this boundary": region_index +=
1
    region ← names[region_index]                    # N boundaries
-> N+1 regions
    region_boundary_dist_um ← distance to the closest boundary
overall
```

Description backends (`_make_desc_fn`)

Four interchangeable backends, selected by `backend_config["backend"]`: - **`rule`** (default, always available, offline): `_rule_based_description` — a template assembling shape (from sphericity/elongation), surface texture (from solidity), branching, and the morphotype tag into one sentence. - **`ollama`**: POSTs the same statistics, formatted into `_STATS_PROMPT`, to a local Ollama server's `/api/generate`. - **`openai`** / **`claude`**: same prompt to the OpenAI Chat Completions or Anthropic Messages API respectively, using a key from `_secrets.py` (§2). All three network backends catch their own request errors and return an inline `"[Backend error: ...]"` string rather than raising — one label's description failure never aborts the whole run.

`_widget.py` — Tab 4 handlers

Per-column checkboxes: `_COL_GROUPS` maps one checkbox to multiple CSV columns where they're conceptually one unit (e.g. `centroid_vox` → all three `centroid_z/y/x_vox` columns; `bbox_vox` → all six bbox corner columns; `nn_1st`/`nn_2nd` → the label+distance pair each). Validation warns (doesn't crash) if brain-region or intensity columns are checked but no Shapes/Image layer is selected, and just skips those columns.

`_refresh_stats_layers()`: on every napari layer add/remove event, repopulates every layer combo in the plugin (Statistics' Image/Shapes, Tab 3's Signal/Labels/Brain-mask selector, Cellpose-SAM's Signal and Brain-mask, Score Against GT's pair), preserving the current selection across the refresh (`blockSignals()` guard), then runs the auto-fill rules (§9 "Layer resolution") for combos that are still empty.

`_on_generate_stats()`: measures `_active_labels_layer()` (the selected Labels layer, else the topmost). Takes the scale from `self._state["metadata"]` first (the value Tab 1's Open button set), falling back to the Labels layer's own napari `scale` (or the active Image layer's, if the Labels layer has none / (1,1,1)) only if no stack metadata

is on record — a real historical bug had this backwards (defaulting to the Labels layer's own possibly-unset scale), silently making `centroid_um`/`volume_um3` identical to their voxel-unit counterparts whenever a labels `.tif` was loaded without its own embedded scale. Extracts region-boundary polylines from the selected Shapes layer via `_extract_region_lines_um()` (accepts both `"line"` and multi-vertex `"path"` shape types), calls `compute_stats()` in a background thread, adds `is_volume_outlier` (`volume_vox` above config `max_volume_recommended_vox` or below `min_volume_recommended_vox`; an unset bound flags nothing), filters the DataFrame down to only the checked columns, and writes `<output>/<opened file stem>_statistics.csv` (the Labels layer's name if no file was opened). Statistics only ever *reads* those bounds; they are written exclusively by the GT Toolkit Tuning Tool (§8).

---

7. Stage 5 — AI Tools

UI: Tab 5 — AI Tools, a manual MONAI-Training/Cellpose-SAM-Training switch (not auto-detected — no per-layer signal to key off here), gated informationally (not blocked) on GPU/VRAM (§2's `_gpu_check.py`). **Goal**: build and train the two models Stages 1–2 depend on, and produce the ground-truth data that trains them.

`_gt_annotation.py` — polygon-based GT annotation

Plain viewer-taking functions on the plugin's own shared viewer, rather than a second `napari.Viewer()`: the target Image layer is passed explicitly (not "first Image layer in the list," a real risk once Tabs 1–3 have added other layers); the `brain_polygons` Shapes layer is auto-created on demand; `generate_masks()` leaves every other layer in the viewer untouched.

```
resample_polygon_preserve_order(pts, n_points=96):
    walk the closed polygon's own perimeter by cumulative arc
length,
    resample to exactly 96 evenly-arc-spaced points — gives every
    hand-drawn polygon, regardless of how many vertices were
clicked,
    the same point COUNT and roughly consistent point
CORRESPONDENCE
    for the interpolation step below
```

```
is_polygon_clockwise(yx) / standardize_polygon_direction(...):
    shoelace-formula signed area — every key-slice polygon is
forced to
    the SAME winding direction as the first (reference) one, since
    point-to-point interpolation between two oppositely-wound
polygons
    would twist through the middle instead of sweeping smoothly

interpolate_shapes(viewer, image_layer):
    by_z ← {slice: resampled 96-point polygon} for every hand-
drawn key polygon
    standardize every polygon's winding to match by_z's own first
slice
    for every Z slice in the volume:
        if a key polygon exists exactly here: use it as-is
        elif slice > PROPAGATION_SLICE (default 90) and
propagation is on:
            reuse the PROPAGATION_SLICE polygon UNCHANGED (just
relabel its
            own Z) — for the common case where the brain
silhouette stops
            changing meaningfully past a certain depth, so no
further
            hand-drawn key slices are needed there
        else:
            find the nearest key slice below and above this one
            linearly interpolate every one of the 96 point-pairs
between
            them by the fractional Z position (point i of the
lower
            polygon -> point i of the upper polygon)
    → a new "brain_polygons_interpolated" Shapes layer, one
polygon per slice

generate_masks(viewer, image_layer, input_path):
    brain_mask ← rasterize every interpolated polygon via napari's
own
                 Shapes.to_labels(), thresholded > 0
    optional median_filter along Z (window 3) to smooth slice-to-
slice
        rasterization jitter
    skin_mask ← 1 - brain_mask
    brain_only / skin_only ← image x each mask
    save brain_mask/skin_mask/original/brain_only/skin_only .tif +
```

```
both
        polygon .npz files under <input>/<stem>/ (the plugin's own
        output-folder convention)
```

`_xzyz_patches.py` — 2D orientation-slice crop generation (current training methodology)

Generates the crop set the plugin's Cellpose-SAM training uses: random 2D crops in all three orientations (the only crop extractor the plugin ships).

```
stretch_z(slice_2d, anisotropy): bilinear zoom of the Z axis by
    anisotropy — brings a Z-cross-section (XZ or YZ) to roughly
the same
    pixel scale as a native XY slice, since this project's voxels
are
    strongly anisotropic
stretch_z_mask(...): the same zoom, nearest-neighbor (order=0) —
integer
    labels must never be blended/interpolated

_random_crops(img_slice, gt_slice, crop_size, n_crops, rng,
min_gt_pixels):
    repeat (up to 30x n_crops attempts):
        pick a random GT-positive pixel as an anchor
        place a crop_size x crop_size window with a random offset
so the
            anchor lands somewhere inside it (not necessarily
centered)
        keep it only if it contains >= min_gt_pixels cell pixels
(label > 0),
            and its own top-left corner hasn't already been used

_anchored_crops(img_slice, gt_slice, anchor_mask, ..., accept):
    same placement rule, anchored on anchor_mask pixels;
accept(gt_crop)
    decides whether a window is kept

_skin_crops(get_slice, skin slice ids, ..., n_boundary,
n_skin_only, touch_px=3):
    for each slice holding skin (-1), until both quotas are
filled:
        boundary: anchor on cell pixels within touch_px of skin
(3x3 dilation
```

```
            x touch_px); keep a window with >= min_gt_pixels cell
pixels
            and some skin; index = 100 + k
        skin-only: anchor on skin pixels; keep a window with no
cell pixel
            and >= min_gt_pixels skin pixels; index = 200 + k

generate_xzyz_patches(image_path, gt_path, out_dir, anisotropy,
crop_size=512,
                      ncrops_per_slice=5,
max_per_orientation=320, ...,
                      skin_crops=True, boundary_fraction=0.20,
                      skin_only_fraction=0.10, touch_px=3):
    load the GT as is (skin -1 kept for sampling); skin crops only
if
        skin_crops and the GT has a -1 voxel
    for each orientation in (XY native, XZ stretched, YZ
stretched):
        shuffle every slice of that orientation holding a cell
(label > 0)
        for each slice (until max_per_orientation total crops
collected):
            draw up to ncrops_per_slice random crops from it
        cap the COLLECTED list to max_per_orientation, THEN write
files
            (not an early-exit mid-slice that could overshoot the
cap)
        if skin crops: shuffle the slices holding skin with a
second
            generator (seed + 1, so the cell crops never change)
and collect
            round(boundary_fraction x n_cell_crops) boundary +
            round(skin_only_fraction x n_cell_crops) skin-only
crops
        save <orientation>_<slice:03d/04d>_<index:02d>.tif +
_masks.tif
            pairs; every mask is written with negative labels as 0
            (background for Cellpose) -- the GT file is never
changed
```

`_crop_truncation.py` — cleaning incidental-neighbor truncation

A crop framed around one target cell often also grazes the corner of a different, nearby cell purely by chance, keeping a valid-looking but tiny sliver whose visible centroid is nowhere near that cell's true center — a genuinely wrong training target for Cellpose's flow-vector loss (which points every voxel toward its own object's true centroid). This is the exact fix applied by hand to this project's real training data before being ported into the plugin.

```
clean_crop_truncation(crop_dir, gt_full_path, anisotropy,
threshold=0.9):
    back up crop_dir first (skip if a backup already exists —
never
        clobbers a still-valid backup with already-modified files)
    for each <stem>.tif/<stem>_masks.tif pair (xy/xz/yz_NNN_NN
naming):
        true_slice ← the SAME stretch_z_mask() transform
generate_xzyz_
            patches() used at generation time, applied to the
fish's own
            full GT volume at this crop's source slice index
(cached —
            many crops share a source slice)
        for each label present in the crop's mask:
            visible ← pixel count in the crop
            true_count ← pixel count in the true full-slice cross-
section
            if visible < threshold x true_count: zero this label
in the
                crop (an incidental neighbor caught mid-
truncation) —
                the crop's own INTENDED target cell is essentially
never
                affected, since it's already near-complete by
construction
        overwrite the mask file only if something was actually
zeroed
```

`_combine_crops.py` — pooling one fish's crops into a shared training folder

Symlinks one fish's own `generate_xzyz_patches()` output into a single shared folder every fish's crops get pooled into, so a multi-fish training run's own `--data_dir` can point at one place regardless of how many fish have been added to it.

```
combine_crop_folder(fish_crop_dir, combined_dir,
fish_stem=None, ...):
    fish_stem ← given, or fish_crop_dir's own parent folder name —
the
        fish's full, already-unique data-folder name (e.g.
        "NT39-3dpf-Crispr-ctrl-D1F4_2024-09-05_15.38.01"), NEVER a
short
        Dish/Fish tag like "D1F4" alone — that shorthand repeats
across
        different experiments/ages in this project (two genuinely
        different fish are both "D1F4")
    pairs ← find_crop_pairs(fish_crop_dir) (reuses
_crop_truncation.py's
        own xy/xz/yz_NNN_NN pair-matching, so that logic lives in
one
        place)
    for each (image, masks) pair:
        link_path ← combined_dir / "<fish_stem>_<original
filename>"
        if link_path already exists or is already a symlink: skip,
            count as "already existed" (never overwritten)
        else: create a real OS symlink to the source file (not a
copy)
    return counts: pairs found, links created, already existed
```

Safe to call again later as new fish become available — an existing symlink is never touched, so re-running after a new fish's own crop folder is ready only ever adds that fish's new links.

`_branch_calibration.py` — measuring real branch radius from GT

Recalibrates `train_xzyz.py`'s `branch_radius` parameter (the erosion-survival-distance threshold the branch-weighted training loss uses to decide "this pixel is part of a thin branch") from actually-measured GT morphology instead of a frozen guess.

```
measure_branch_diameters(gt_labels, scale_zyx, pad, min_volume):
    for each labeled cell (cropped to its own padded bbox):
        edt ← anisotropic distance_transform_edt(binary_mask,
sampling=scale_zyx)
        skel ← skeletonize(binary_mask)                    #
topological thinning
        skel_valued ← edt VALUES baked onto the skeleton's own
nonzero
            voxels (a real skan API gotcha: Skeleton(...,
source_image=...)
            does NOT feed skan.summarize()'s own mean-pixel-value
column —
            that column reads the skeleton_image array's own
values, so
            the EDT has to be baked IN, not passed as a separate
argument)
        bd ← skan.summarize(Skeleton(skel_valued,
spacing=scale_zyx))
        diam_um ← 2 x bd's mean-pixel-value per branch segment
    → pooled diam_um / length_um across every cell

recommend_branch_radius(gt_labels, scale_zyx, xy_scale=None):
    stats ← measure_branch_diameters(...)
    tip_diam_um ← the thinnest QUARTILE of all measured diameters
                  (distal branch tips — what branch_radius exists
to protect)
    tip_radius_um ← mean(tip_diam_um) / 2
    recommended_px ← round(tip_radius_um / xy_scale)   # µm ->
pixels at the
                       crop's own XY resolution, directly
comparable to
                       train_xzyz.py's own branch_radius parameter
```

An approximation, not an exact reconstruction — the training-time threshold is a Chebyshev (chessboard) erosion distance in pixels; this measures a true Euclidean physical radius and converts. Close enough to calibrate a threshold, acknowledged in the docstring as not identical.

`_training_jobs.py` — cross-platform detached process management

Both MONAI (`train.py`, hours–days) and Cellpose-SAM (`train_xzyz.py`, ~20h) training runs must survive napari closing and be re-discoverable after a full restart. Built on primitives that behave identically on Linux/Mac/**Windows** (this plugin ships a native Windows install path, so a tmux-only design was rejected): a detached `subprocess.Popen` + PID tracking via `psutil`, stdout/stderr to a plugin-controlled log file the GUI tails.

```
launch_detached(argv, cwd, log_path, conda_env, notify=None):
    if notify is given:
        write a small STANDALONE supervisor .py file (stdlib-only
—
            subprocess/smtplib/re, no dependency on this package
being
            importable) that: runs the real "conda run -n <env>
<argv>"
            command (same log_path the GUI already tails live) ->
waits
            for it to exit -> parses the log for the best-
checkpoint
            metric -> emails a completion report via SMTP_SSL
        the SUPERVISOR itself is what's launched detached, not
argv
            directly — so the email still fires even if napari
closes
            and never reopens before the job finishes.
kill_process_tree
            needs no special case: the real training process is a
            descendant of the supervisor, so killing the whole
tree (a
            manual Stop click) kills the supervisor before it
reaches
            its own email step, correctly sending no "stopped"
email.
    else:
        Popen(["conda","run","-n",env,"--no-capture-
```

```
output",*argv],
              stdout/stderr -> log_path,
              start_new_session=True on POSIX (setsid) /
              CREATE_NEW_PROCESS_GROUP|DETACHED_PROCESS on
Windows)
    return pid

is_running(pid) / kill_process_tree(pid): psutil-based liveness
check /
    recursive terminate-then-force-kill of pid + every descendant
(conda
    run spawns a child python process, so killing only the top PID
would
    leave training running)

tail_log(log_path, n_bytes=8192): last N bytes only, bounded
regardless
    of how many days/GB the log has grown to

patience_exceeded(log_path, metric_cfg, patience):
    parse every (epoch, metric_value) the log's own regex matches
        (MONAI_METRIC: "Epoch N Summary:" ... "Full-brain Dice: X
         [MODEL SELECTION]", DOTALL, higher-is-better;
CELLPOSE_METRIC:
         "N, train_loss=..., test_loss=X" on one line, lower-is-
better —
         the ONLY thing that differs between the two scripts'
early-
         stopping is this metric config, the counting logic is
shared)
    best ← the best-value checkpoint logged so far
    checkpoints_since_best ← how many logged checkpoints have
passed
        without a new best
    exceeded ← patience > 0 and checkpoints_since_best >= patience
    # patience counts CHECKPOINTS (the plugin's own observable
cadence
    # from the log), not epochs — each script's own internal
epoch/val
    # cadence differs and doesn't matter here

write_best_checkpoint_pointer(models_dir, model_name, best_epoch):
    writes "<model_name>_best_recommended.txt" naming the
checkpoint file
```

```
    — a plain TEXT pointer, not a copy (checkpoints are 100s of
MB) and
    not an OS symlink (needs elevated privileges/Developer Mode on
    Windows) — resolves identically cross-platform with zero
special
    permissions. Only needed for Cellpose-SAM, which has no built-
in
    best-tracking of its own (MONAI's train.py already auto-saves
its
    own best_model_fullstack.pth).
```

`_ai_tools.py` — argv builders

Pure argument-list construction (`build_prepare_data_argv` / `build_monai_train_argv` / `build_cellpose_train_argv`) for the three training scripts, which ship *inside the installed package* (`training_scripts/`, declared as package data) so they travel into `site-packages` on both editable and normal installs. The script paths are overridable only through config keys (`monai_prepare_script_path`, `monai_train_script_path`, `cellpose_train_script_path`); there is no browse button. The conda env each runs in comes from `monai_conda_env` (default `zf-microglia-ai`, used for prepare_data and MONAI training) and `cellpose_conda_env` (default `cellpose`).

`_widget.py` — Tab 5 (AI Tools) handlers

GT Annotation section: buttons wrapping `interpolate_shapes()`/ `generate_masks()` directly — synchronous (both are fast: pure numpy/ shape rasterization, no model inference), errors shown in the status label rather than crashing.

Extract X/Y/Z Patches (`_on_xz_run()`): background-thread wrapper around `generate_xzyz_patches()` (anisotropy = Z µm / XY µm from its own scale fields), passing “Add skin crops” and the two percentage fields as `skin_crops`/`boundary_fraction`/`skin_only_fraction`, followed in the same thread by `clean_crop_truncation()` when “Clean truncated labels” is ticked (the default; threshold = “Min visible fraction”, default 0.90). The status line reports the cell crops, the boundary and skin-only counts (or that the GT has no skin label), and the cleanup. No cancel button.

Combine Crops into Shared Training Folder (`_on_cc_run()`): wraps `combine_crop_folder()` (fish_stem left to its default, the crop folder's parent name); on success stores the combined folder in config and fills Train Cellpose-SAM's Data dir.

Calibrate branch_radius (from GT): background-thread wrapper around `recommend_branch_radius()`, gated by a hidden "This is verified ground truth" checkbox — run here it only reports; only the GT Toolkit Tuning Tool (§8) ticks that box, and only then is the shared `branch_radius` ceiling (`_update_gt_history(..., mode="max")`) updated and applied.

Train MONAI U-Net / **Train Cellpose-SAM** buttons: build the argv via `_ai_tools.py` (MONAI's own `--patience` is always passed as 999999 so only the GUI's patience check stops a run), call `launch_detached()` with the log at `<model dir>/gui_launch_<session>.log` (MONAI) or `<data dir>/logs/gui_launch_<session>.log` (Cellpose-SAM); poll `is_running`/`tail_log`/`patience_exceeded` on an 8-second `QTimer` while napari stays open, and **reconnect automatically on napari reopen** by reading the persisted PID + log path from config — a dead PID found on reopen finalizes immediately (best checkpoint reported, Cellpose-SAM pointer written, stale PID cleared), so a job that finished while napari was closed is reported the moment napari reopens. "Stop Training" calls `kill_process_tree`. When Tab 6's email switch is on, `_build_notify_cfg()` threads a `notify` dict (SMTP settings, password via `_secrets.py`) through to `launch_detached`.

---

8. Stage 6 — Sweeps & Utilities

UI: Tab 6 — Sweeps & Utilities. **Modules**: `_sieve.py` (generic coarse-to-fine engine, already covered in §2), `_brain_sweep.py`, `_pixel_sweep.py`, `_krendl_sweep.py`, `_epoch_sweep.py` (the GT-verified parameter sweeps; `_pixel_sweep.py` holds two), `_gt_score.py` (the whole-fish scorer), `_gt_package.py` (packaging for external manual correction), `_gt_toolkit.py` (single-folder auto-discovery behind the GT Toolkit Tuning Tool). **Goal**: check a pipeline parameter against real ground truth and, once confirmed, recalibrate the "recommended" values quoted throughout §3–§5.

`_gt_score.py` — the whole-fish scoring methodology

`score_against_gt(pred_labels, gt_labels, iou_threshold=0.5)` is the project's whole-fish instance-matching methodology, behind Tab 6's Score Against GT (the sweeps use their own simpler best-IoU match per cell, `_epoch_sweep._best_gt_match`, or mask-level Dice):

```
pred_info, gt_info ← per-object volume/centroid/bbox (_get_info
from _cellpose_seg.py)
build a (n_gt x n_pred) IoU cost matrix, but only for pairs whose
BBOXES
    actually intersect (IoU is necessarily 0 otherwise — keeps
this fast
    even for dozens of objects, without ever missing a real match)
row_ind, col_ind ← linear_sum_assignment(-iou_matrix)   #
Hungarian algorithm,
                     the GLOBALLY optimal one-to-one matching, not
a greedy
                     nearest-first assignment
for each assigned (gt, pred) pair:
    if IoU >= iou_threshold: count as TP, record IoU/Dice/
size_delta
every unmatched GT object → FN;  every unmatched pred object → FP
score ← TP - 0.5 x (FP + FN)                    # this project's
own established metric
mean_iou / mean_dice ← averaged over TP matches only
```

Verified against this project's own historical result tables (e.g. `TP=34/FP=8/FN=5 → Score=+27.5`).

`_brain_sweep.py` — MONAI Threshold x Erosion (Tab 1's own mask)

The cheapest of the sweeps: `predict_probability()` (the expensive network pass) runs **once**; every candidate `(threshold, erosion)` combination is then a cheap re-threshold via `postprocess_probability()` + a `binary_erosion`, scored as a whole-volume Dice/IoU/precision/recall against a hand-corrected GT brain mask (from GT Annotation, §7) — no per-cell matching needed, since brain segmentation is one global object.

```
pred_prob ← predict_probability(volume, model_path, device)   #
once, or
            reused via precomputed= across sieve stages
```

```
for each threshold:
    raw_mask ← postprocess_probability(pred_prob, threshold)
    for each erosion:
        mask ← binary_erosion(raw_mask, erosion) if erosion > 0
else raw_mask
        score whole-volume Dice/IoU/precision/recall vs GT
best_point ← the (threshold, erosion) with highest Dice
```

`_pixel_sweep.py` — the Pixel Classifier route's two sweeps + GT-measured floors

Deliberately does **not** re-run MONAI inference per grid point (neither Signal Erosion nor BG Threshold change what MONAI predicted) — takes a pre-computed, un-eroded `brain_mask.tif` as input, making the whole sweep `create_labels()`-bound (minutes, no GPU needed) rather than GPU-inference-bound.

`run_pixel_sweep(image_path, brain_mask_path, gt_labels_path, bg_thresholds, erosions, ...)`:

```
cells ← find_complex_cells(gt_labels, n_cells)     # most skeleton
branches
        first, NOT by volume — a large cell is often simple/
amoeboid,
        while a genuinely branchy cell stresses the pipeline more
for each cell: crop image+brain_mask+GT to its own padded bbox
once
bg_max ← the GLOBAL background estimate (histogram over the whole
          volume's in-brain pixels) — computed once, since it's a
          genuinely global quantity, independent of Signal Erosion
for each bg_threshold:
    thresh ← bg_max + data_range x (bg_threshold / 100)
    for each erosion:
        for each cell's own crop:
            signal_mask ← erode_signal_2d((image > thresh) &
brain_mask, erosion)
                            # erodes what SURVIVED the threshold,
not the
                            # probe mask itself — matches
remove_global()
            pred_labels ← create_labels(image x signal_mask,
sigma_xy, sigma_z,
                                        min_volume,
min_hole_size)
```

```
            score ← best-IoU match of any predicted object against
this cell's GT
    average IoU/Dice across all N cells at this grid point
best_point ← highest average IoU
```

`run_sigma_sweep(...)` — the Smooth σXY/σZ counterpart, roles reversed: BG Threshold/Signal Erosion held fixed, σXY/σZ swept. Cheaper per point than `run_pixel_sweep`, since σ only affects `create_labels()`, not the background-threshold crop-building step (computed once per cell, reused across every σ combination).

`min_volume_from_gt(gt_labels)`: the smallest true voxel volume among GT-labeled cells, measured per fish rather than one constant shared across every fish regardless of how its real microglia actually run. **`min_hole_size_from_gt(gt_labels, min_hole_size_to_trust=5)`**: the smallest *real* internal hole any hand-corrected GT cell actually has (a human deliberately left it unlabeled) — holes below a trust threshold are discarded first, since real GT showed a sharp bimodal split (1–2 voxel annotation slips vs. a cluster of 400+ voxel real structural gaps, nothing in between); trusting every reported hole collapsed the recommendation to 0. **`min_intercell_gap_um(gt_labels, scale_zyx, pad_um=15)`**: smallest real physical surface-to-surface gap between any two distinct GT cells — a safety ceiling for Krendl safe-merge's `max_gap` (never bridge a gap that could be two real cells), computed per-cell-bbox rather than one full-volume distance transform (which timed out past 100s on a real fish).

`_epoch_sweep.py` — GT-verified Cellpose-SAM checkpoint sweep

Automates the bbox-restricted GT-IoU methodology developed by hand across this project's training rounds to confirm a checkpoint pick (from `test_loss`, a proxy metric) against real GT.

```
find_complex_cells(gt_labels, n_cells):
    rank every GT cell by (n_branches, -sphericity) — same
skeleton/
    surface-area code as Statistics (§6), so this ranking matches
what's
    reported elsewhere in the plugin
bbox_crop(image, gt_labels, label_id, pad_z=15, pad_xy=40): crop
to one
    cell's own padded bbox — do_3D cost scales with crop size, so
this
    turns a full-fish sweep (~hours per checkpoint) into a small
```

```
handful
    of seconds-scale crop inferences

run_epoch_sweep(image_path, gt_labels_path, models_dir,
model_name, epochs, ...):
    crop each of the N complex cells once
    for each candidate epoch:
        load that checkpoint ONCE (not once per cell)
        for each cell's crop: raw do_3D inference (no GMM/Krendl —
just
            best-IoU-match the single predicted object against
this cell's GT)
    average IoU/Dice per epoch across all N cells
best_epoch ← highest average IoU
```

`pick_sweep_epochs(recommended_epoch, save_every, available_epochs, n_below=2, n_above=2)`: centers the candidate epoch list on the `test_loss`-recommended epoch, stepping by the training run's own checkpoint interval, clipped to whatever actually exists on disk.

`_krendl_sweep.py` — Cellprob + real-data merge / Min size / Large-contact calibration

The most involved sweep module: one cellprob sweep plus three measure/recommend pairs, chained by Tab 6's **Calibrate Cellprob** + **Krendl Merge Parameters** + **Min size** + **Large-contact** (`_on_kr_run_sweep()`, see the end of this section). `gt_min_from_labels` is simply `_pixel_sweep.min_volume_from_gt` under a second name — `gt_min` and the Pixel Classifier's `min_volume` are the same quantity (the smallest GT cell's voxel count).

`run_cellprob_voxel_sweep(volume, gt_labels, model_path, cellprobs, anisotropy, gpu, min_size, min_hole_size, niter, progress_cb, cancel_event, precomputed)` — scores cellprob on **pure voxel-level Dice/IoU/precision/recall** against binarized GT (instance identity ignored entirely), deliberately independent of `max_gap`/`min_contact`/`large_contact` so the cellprob pick isn't entangled with not-yet-calibrated merge parameters — mirrors `_brain_sweep.run_brain_sweep()`'s mask-level approach for MONAI Threshold:

```
model, dP, cellprob_map, shape ← precomputed or
predict_flows(...)   # ONE network pass
for each cellprob (cancel_event checked between values):
    masks ← masks_from_flows(..., cellprob, min_size,
```

```
min_hole_size)
    results[cellprob] ← voxel Dice/IoU/precision/recall of (masks
> 0) vs (gt > 0)
best_cellprob ← highest Dice
return {results, best_cellprob, cancelled, precomputed}   #
precomputed lets sieve
                                                            #
stages reuse the pass
```

`measure_merge_params_from_prediction(cp_masks, gt_labels, scale_zyx, min_overlap_vox=5, search_pad_um=3.0)` — measures `max_gap`/`min_contact` from a real raw (pre-GMM, pre-Krendl) prediction rather than pure-GT geometry (which has a real ambiguity: is the smallest gap between two GT cells real biology, or just how an annotator drew the shared boundary?). Sidesteps that by using GT only to *label* which raw fragments belong to which real cell, then measuring the network's own actual fragmentation:

```
assign every raw fragment to whichever GT cell it overlaps MOST
(drop as
    noise if the best overlap is < min_overlap_vox)
"should_merge" samples: for every GT cell with >= 2 assigned
fragments,
    each fragment's gap/contact to its NEAREST same-cell sibling
only —
    not all pairwise combinations (krendl_safe_merge() only ever
needs to
    bridge to the nearest neighbor, iteratively; all-pairs would
badly
    inflate the "should merge" gap distribution with meaningless
far-apart
    pairs within one large sprawling real cell)
"should_not_merge" samples: fragment pairs assigned to DIFFERENT
GT cells
    that end up close (bbox-proximity pre-filtered) — an
unambiguous
    safety-ceiling data point, since GT already confirms these are
    genuinely separate cells
```

`recommend_merge_params(merge_stats_list)` — turns those pooled samples into one recommended `(max_gap, min_contact)` **pair**, jointly optimized against the REAL OR-combined decision rule

(`krendl_safe_merge` merges if `gap <= max_gap` OR `contact >= min_contact`), not two independently-optimized single-criterion thresholds:

```
every OBSERVED (gap, contact) value becomes a candidate threshold
(exact,
    not a coarse grid)
for every candidate (max_gap, min_contact) pair:
    missed ← should-merge samples where BOTH gap > max_gap AND
contact < min_contact
              (computed for the whole grid in one matrix multiply,
               _count_gap_gt_and_contact_lt, not a triple-nested
loop)
    false ← should-not-merge samples where NEITHER criterion would
have
             excluded them (i.e. the pair WOULD wrongly merge
them)
    total_error ← missed + false
best ← the (max_gap, min_contact) pair minimizing total_error
```

Optimizes for **fewest total manual corrections**, not zero risk of one error direction: an over-merge is trivially fixable with Split Label, an under-merge with Join Labels (§5), so neither failure is silent or catastrophic. Returns `{max_gap_um, contact_vox, missed_merges, false_merges, n_should_merge, n_should_not_merge}` (both values `None` when there are no samples).

`measure_min_size_from_prediction(cp_masks_unfiltered, gt_labels, min_overlap_vox=5)` — on masks formed with `min_size=0` (any positive min_size would already have deleted the fragments to be measured): every fragment whose best GT overlap is ≥ `min_overlap_vox` counts as a **real** fragment (its size recorded), every other fragment as **noise**. **`recommend_min_size(stats_list, miss_weight=1000)`** pools the sizes across fish and tries every observed size as the threshold, minimizing `1000 × real fragments discarded + noise fragments surviving` (ties → the largest candidate) — asymmetric on purpose, since a discarded real fragment can't be restored while surviving noise still meets three later filters.

`measure_large_contact_from_prediction(post_safe_merge_masks, gt_labels, min_overlap_vox=5, bbox_margin_vox=2)` — on masks after GMM + safe-merge (exactly what `large_contact_merge()` receives): for each GT cell still split into ≥2 objects, each object's **largest** contact with a same-cell sibling is a should-merge sample; the contact between bbox-close objects assigned to **different** GT cells is a should-not-merge

sample. **`recommend_large_contact(stats_list, miss_weight=1.0)`** tries every observed contact as the threshold, minimizing missed + false merges (symmetric; ties → the largest candidate).

The widget's five-step chain (`_on_kr_run_sweep()`):

```
min_hole_size ← min_hole_size_from_gt(gt_labels)            # used
by every mask formation below
1. run_cellprob_voxel_sweep (sieve-narrowed if on), Tab 2's Min
size
   — if cancelled or no winner, stop here
2. raw_cp_masks ← masks_from_flows(cached flows, best_cellprob,
Tab 2's Min size)
3. merge_stats ←
measure_merge_params_from_prediction(raw_cp_masks, gt, scale)
4. unfiltered ← masks_from_flows(cached flows, best_cellprob,
min_size=0)
   minsize_stats ← measure_min_size_from_prediction(unfiltered,
gt)
5. post ← krendl_safe_merge(gmm_cleanup(raw_cp_masks), this fish's
merge
          recommendation (or Tab 2's current values if none),
gt_min_from_labels(gt))
   lc_stats ← measure_large_contact_from_prediction(post, gt)
apply (only when its hidden GT box was ticked by the GT Toolkit):
   Cellprob ← _update_gt_history("cellpose_cellprob", fish, best,
"mean")
   max_gap/min_contact, Min size, Large-contact ← recommend_*()
over the
       samples pooled across every fish (_update_*_stats_history),
each left
       unchanged if still None
```

`_gt_package.py` — packaging for external manual correction

`build_gt_package(stem, out_dir, image_path, corrected_masks_path, raw_cellpose_masks_path=None, brain_mask_path=None, raw_signal_path=None, autocorrect_report_path=None, scale_zyx, guide_path=None, progress_cb)` — automates a manual packaging step repeated by hand for every fish sent out for GT correction, into `<out_dir>/<stem>_GT_package/`: first deletes any file it is about to write from a previous build in the same package folder (so an optional input left empty never ships a stale copy), then copies the source image (as

<stem>_brain_only_ExtRm.tif), the corrected masks (as <stem>_cp_corrected.tif, the correction starting point), the raw pre-merge masks (<stem>_cp_masks_3D.tif, reference only), the brain mask (<stem>_brain_mask.tif) and the raw unmasked signal channel (re-written zlib-compressed as <stem>_original.tif, rejected unless 3D) — both needed for the reviewer to use Protect Skin as Label — the auto-correct report (<stem>_autocorrect_report.txt), and GROUND_TRUTH_CREATION_GUIDE.md (DEFAULT_GT_GUIDE_PATH = the repo's docs/ copy unless overridden; missing → error), plus a lightweight per-cell CSV (_cell_statistics_csv — label/volume/centroid/bbox only, largest first, deliberately not the full Tab 4 output), all zipped into <out_dir>/<stem>_GT_package.zip. The widget passes _get_layer_scale() as the CSV's scale.

_gt_toolkit.py — single-folder auto-discovery ("GT Toolkit Tuning Tool")

Given one fish's original source file (or its already-created output folder), locates every artifact this project's established naming convention produces there and decides which of the above sweep/calibration steps can actually run without the user hunting down and browsing to each file by hand.

```
SUFFIXES ← the fixed map of {key: filename suffix} this plugin's
own
           _output_dir() convention produces (_original.tif,
_brain_mask.tif,
           _brain_only_ExtRm/NoBG/RndFill.tif, _cp.tif,
_cp_krendl.tif,
           _cp_krendl_ac.tif, _cp_krendl_ac_snd.tif,
_GROUND_TRUTH.tif,
           _statistics.csv, _autocorrect_report.txt) — Cellpose-
SAM's own naming is cumulative and
           self-documenting: _cp_krendl_ac_snd.tif's name itself
says it
           went through Krendl, then auto-correct, then sanding

discover_fish_files(source_path):
    resolve_fish_folder(...) ← accepts the folder itself, any file
inside
        it, or the original raw source file sitting beside it
    for each SUFFIXES key: an exact <stem><suffix> match, else the
single
        result of a same-suffix glob if exactly one exists
```

```
(tolerates the
        folder name and a file's own stem drifting apart
slightly), else None
        — except _autocorrect_report.txt, where several per-layer
reports
        are normal: the most recently modified match wins

best_corrected_masks(found): the most-advanced stage present,
priority
    sanded > auto-corrected > Krendl-only (never the raw "cp"
stage,
    which is always kept as a separate reference file)

step_preconditions(found): {step_key: (can_run: bool, reason:
str)} for
    each of the 5 folded-in steps (monai/bg/sigma/cellprob/
branch_radius)
    — monai: _original + brain_mask_is_hand_corrected(); bg:
(original or
    ext_rm) + brain_mask + ground_truth; sigma: the same + no_bg;
cellprob:
    ext_rm + ground_truth; branch_radius: ground_truth.

brain_mask_is_hand_corrected(found): (ok, reason). Run Skin-
Remover and GT
    Annotation both write *_brain_mask.tif (and *_original.tif),
so the
    name proves nothing. GT Annotation's Generate Masks also
writes
    <stem>_brain_polygons_interpolated.npz (SUFFIXES key
"brain_polygons")
    in the same call, so:
        no brain_mask or no polygons file          -> not hand-
corrected
        polygons stem != brain_mask stem           -> not hand-
corrected
        mask mtime - polygons mtime > 300 s        -> overwritten
later

(_HAND_MASK_MAX_LAG_S)
        otherwise                                  -> hand-
corrected
```

`_widget.py` — Tab 6 (Sweeps & Utilities) handlers

Every sweep tool shares: a **Sieve checkbox** (§2's `_add_sieve_controls()` — coarse full-range pass, then 2 narrowing stages around the winner; not on Verify Best Epoch) wired to `run_sieve()`; a one-shot **"This is verified ground truth"** checkbox (off by default, un-ticked by the tool's own poll after every run) gating whether the result is allowed to apply to a live slider/config value or just reported — hidden on the MONAI/BG/Sigma/Cellprob sweeps (and Calibrate branch_radius), visible only on Verify Best Epoch; a **Stop** button wired to a `threading.Event` checked between grid points; and, once a value is applied, a call to `_update_gt_history(config_key, fish_key, value, mode)` (or one of the pooled `_update_*_stats_history` variants) — the shared cross-fish mechanism every "recommended" value in this plugin is tracked through (§9).

GT Toolkit Tuning Tool (`_on_gtk_scan()` / `_on_gtk_run()` / `_gtk_poll()` / `_on_gtk_stop()`): **Scan** runs `discover_fish_files()`, shows `format_discovery_report()` and fills every folded-in tool's path fields (`_gtk_autofill()`: MONAI image = `_original`, BG/Sigma image = `_original` or `_ExtRm`, Cellprob image = `_ExtRm`, GT = `_GROUND_TRUTH`, plus Calibrate branch_radius and the GT-package fields). **Run** scans again, reads and un-ticks its own visible "verified" box, then:

```
step 0 (instant, if _GROUND_TRUTH.tif exists): min/max cell voxel
counts +
    min_hole_size_from_gt(); if verified → _update_gt_history for
    min_volume_vox (min), max_volume_vox (max), min_hole_size_vox
(min),
    applied to the Common Settings values/slider and the
Statistics ceiling
queue ← the steps step_preconditions() allows, in order
        monai → bg → sigma → cellprob → branch_radius (skipped
ones reported)
for each queued step (polled every 500 ms, one at a time):
    tick that tool's hidden GT checkbox to match "verified", click
its Run
    wait until its thread AND its own poll timer have finished
    append its status line to the report
_notify_outer_job is set for the whole queue, so only the toolkit
emails
Stop: cancels the running step, truncates the queue after it
```

Build GT-Correction Package: background-thread wrapper around `build_gt_package()`; browsing **Source image** runs `discover_fish_files()` on it and fills stem, `best_corrected_masks()`, `_cp.tif`, `_brain_mask.tif`, `_original.tif`, the newest `_autocorrect_report.txt` and the output folder (the GT Toolkit's Scan fills the same fields). Missing brain mask / raw signal / report don't block the build; the status line warns what the reviewer will lack.

---

9. `_widget.py` — application shell

At about 12,000 lines and 165 functions/methods, `_widget.py` is both the biggest file in the plugin and the least algorithmically interesting one — the vast majority of it is straightforward Qt widget construction (`_build_ui()`, ~4,200 lines) and thin handler functions that read a few widgets, call into one of the modules covered in §3–§8, and write the result back to a napari layer via the threading pattern in §1. Rather than re-describe every handler a second time, this section covers the parts of the file that are genuinely its own infrastructure — construction, config persistence, cross-fish history, layer resolution, the hang watchdog, and email notification — and then indexes every handler by which pipeline-stage section already documents it.

Class structure

```
_FastDirectLabelColormap(DirectLabelColormap)   — module-level,
see §5
                                                    ("Hide skin
label")
_load_config() / _save_config(data)             — module-level,
plain
                                                    JSON read/
write of
                                                    ~/.config/
napari-zf-
                                                    microglia-ai/
config.json
_add_reliable_spinbox / _add_gt_checkbox /
_add_sieve_controls / _is_valid_cellpose_
checkpoint / _add_recommended_label /
_set_layout_widgets_visible / _make_collapsible /
_wrap_scroll / _sep /
_extract_region_lines_um                        — module-level
```

```
shared
                                                    UI helpers,
see §2/§6

class ZFMicrogliaAIWidget(QWidget):
    __init__(napari_viewer)
    _build_ui()            — constructs every tab's every widget
    _connect_signals()      — wires every button/combo/layer-event
to its handler
    <~150 methods total>    — handlers, one section per tab, plus
shell helpers
```

`__init__` and startup sequence

```
cfg ← _load_config()
cfg ← _secrets.migrate_plaintext_secrets(cfg)   # one-time
upgrade, see §2
model_path ← saved config path, else the bundled default
checkpoint, else
             None — resolved via is_file(), not exists(): Path("")
silently
             normalizes to Path(".") under pathlib, and exists()
is True
             for a directory too, which would otherwise let an
unconfigured
             path pass as "a valid model is loaded"
cellpose_model_path ← saved config only — no bundled default
(project-specific,
             not shipped with the plugin)
self._state ← {model_path, cellpose_model_path, last_file_path,
metadata, config}
self._skin_hidden_state ← None   # see _on_toggle_skin_visibility,
§5
self._notify_settings ← {enabled: False}; _notify_outer_job ←
None   # email, below

_build_ui()
_connect_signals()             # also fires the initial-visibility
calls and
                               # _resume_monai_job_if_active() /
_resume_
                               # cellpose_job_if_active() (§7's
"reconnect
                               # on reopen" mechanism)
```

```
_refresh_layer_info()
_refresh_stats_layers()
_start_recovery_timer()
```

`_build_ui()` — organization, not algorithm

One long method building six `QTabWidget` pages in order (Skin Remover / Create MG Labels / Edit MG Labels / Statistics / AI Tools / Sweeps & Utilities), each wrapped in `_wrap_scroll()` and built mostly from `_make_collapsible()` group boxes. The Tab 6 (Sweeps & Utilities) layout is created first, so groups built inside the other tabs' code (each GT sweep, Score Against GT, Build GT-Correction Package, Email notification, Drift View, Autosave, Calibrate Correct-Label Contrast) register themselves in `self._t5_groups` (+ a category list for the "Show tools for…" filter) and are placed by one `_t5_order` tuple at the end, guarded by an assert that every registered group is listed. There's no real control flow here worth pseudocode — every group follows the same shape: a `QGroupBox`, a `QVBoxLayout`, a slider+`_add_reliable_spinbox()` pair per numeric parameter, a status label, a run button. The one thing worth naming explicitly is the **Common Settings** group (Tab 2): **Min volume** and **Max volume** are deliberately *read-only labels*, not editable sliders — both are empirical GT-measured facts (§8), not tunable knobs; Min hole size and Final min-size fraction are ordinary sliders.

Config persistence

```
_load_config() / _save_config(data):  plain JSON read/write, empty
dict / no-op
                                        on any failure — never
raises, so a
                                        corrupted or missing
config file never
                                        blocks the plugin from
opening
_save_cfg(self, **kwargs):  self._state["config"].update(kwargs);
_save_config(...)
                              — the one method every handler
actually calls;
                                merges rather than replaces, so
one handler's
                                save never clobbers another's
unrelated keys
```

Cross-fish GT-sweep history (`_update_gt_history`, `_update_merge_stats_history`)

The mechanism behind every "Recommended: X" value quoted throughout §3–§8 — a real per-fish history, not a single overwritten scalar (an earlier design either blindly overwrote the target value with whatever one run found, or tracked a single running scalar with no way to revise a re-swept fish's own contribution):

```
_update_gt_history(config_key, fish_key, value, mode):
    history ← config["<config_key>_history"]           #
{fish_key: value}
    history[fish_key] ← value                            # update
in place if
                                                           # this
fish was
                                                           #
already swept
    save history back to config
    return  min(history.values())  if mode == "min"       # never-
rising floor:
                                                           #
min_volume, min_
                                                           #
hole_size, gt_min
            max(history.values())  if mode == "max"       # never-
falling ceiling:
                                                           #
branch_radius
            mean(history.values()) otherwise               # BG
Threshold, Erosion,
                                                           #
Sigma XY/Z, Cellprob,
                                                           #
Large-contact, MONAI
                                                           #
Threshold — each
                                                           #
fish's own local
                                                           #
optimum, no safe
                                                           #
direction to bias
```

`_update_merge_stats_history`, `_update_minsize_stats_history` and `_update_largecontact_stats_history` are the exceptions, storing a whole *sample-list dict* per fish (not a scalar) and re-running `recommend_merge_params()` / `recommend_min_size()` / `recommend_large_contact()` (§8) over the freshly-pooled set from every fish calibrated so far, rather than averaging a per-fish scalar.

Layer resolution

Three different "which layer" resolvers, each deliberately scoped to a different part of the plugin (see §1's "Layer-selection philosophy"): - **`_active_layer()`**: active selection if it's an `Image` layer, else the topmost `Image` layer, else `None` — the plugin's original, still-used-for-Tab-1 fallback. - **`_active_labels_layer()`**: same pattern for `Labels` layers — used by Generate Statistics and Re-run This Cell Only's "Use selected" button. - **`_edit_labels_layer()`**: reads the *explicit* combo the user set in Edit MG Labels' own "Layers and label(s) being edited" section — every Stage-3 tool (§5), Calibrate Correct-Label Contrast and Autosave use this one, never a fallback, so the layer being edited is never guessed. It also forces `n_edit_dimensions = 2` on that layer each call (a guard for napari's own paint/fill tools); `_refresh_stats_layers()` calls it after every layer change for exactly that side effect. - **Auto-fill** (inside `_refresh_stats_layers()`, only for combos still on "None"): anchor = the selected Image layer ending in `_brain_only_ExtRm/_NoBG/_RndFill/_brain_only`, else the topmost such layer; `stem` = anchor minus that suffix. Signal ← the layer named exactly `stem` (the raw channel; never the anchor), Brain mask ← `<stem>_brain_mask`, Labels ← `<anchor>_cellpose_labels` or `<anchor>_labels`, else the selected/topmost Labels layer not ending in `_brain_mask`. Cellpose-SAM's Signal combo ← the first `_ExtRm` layer; its Brain-mask combo ← `<stem>_brain_mask` of the active layer. - **`_raw_channel_layer(name)`** / **`_resolve_skin_signal_image(name, out_dir)`**: strip one `_BG_SUFFIX` and then `_brain_only` from a layer name to find the raw channel — the first returns that open Image layer (used by the chained Cellpose-SAM auto-correct, which requires it); the second, used for skin's own trim, returns that layer's data, else `<out_dir>/<raw>_original.tif`, else `None` (caller falls back to the signal layer) with a note saying which.

`_get_layer_scale()`: metadata from the last Open-button load, else the active layer's own napari `scale`, else `(1,1,1)`. `_output_dir()`: `<original file's parent>/<original file's stem>`, created on first use — the original file being the last one opened with the plugin's Open button, else the first layer whose napari `source.path` is set (cached), else the current working directory (with a printed warning). The one folder

convention every saved output file in the plugin follows (and that §8's `_gt_toolkit.py` reverse-engineers to auto-discover a fish's files). `_current_min_volume()` / `_lo_sweep_sample_params()`: thin config readers so every route/sweep launcher shares one source of truth instead of each hardcoding its own possibly-diverging copy.

The hang watchdog

```
_hang_watch_start(what, after_s=600):
    faulthandler.dump_traceback_later(after_s, repeat=True,
file=sys.stderr)
    # if the operation is STILL running after after_s seconds (and
every
    # after_s after that), every thread's real stack prints to the
    # terminal napari was launched from — turns "is it stuck?"
into
    # "here's exactly which line it's on", instead of only the
last
    # status message (which can stay unchanged through several
later steps)
_hang_watch_stop():
    faulthandler.cancel_dump_traceback_later()
```

Wrapped around Protect Skin as Label, Auto-correct Labels, and the Cellpose-SAM-chained auto-correct stage — the slowest operations in the plugin — with `after_s=1800` (30 minutes).

Email notification

One plugin-wide switch (**Email me when a job finishes or fails**, default on) and one threshold (**only if it ran longer than**, default 10 min) in Tab 6's Email notification panel; no per-tool checkboxes. Two paths:

- **In-process jobs** — every background worker in `_widget.py` is started through `_job_thread(label, target, result, notify_job=None, auto_finish=True)`, which returns an unstarted daemon thread whose wrapper stamps the start time, runs the worker (an uncaught exception becomes `result["error"]`), and — with `auto_finish` — calls `_finish_notify_job()` as soon as it returns, ok unless `result` holds an `"error"` (the one failure convention every worker in the file already follows). Tools whose own poll writes a richer email body (Run Skin-Remover, Cellpose-SAM Segmentation, the Cellprob and Best-Epoch sweeps) pass `auto_finish=False` and

call `_finish_notify_job(thread.notify_job, ok, subject, body)` themselves. A chained stage (Cellpose-SAM → auto-correct → sanding, Auto-correct Labels → sanding) passes the first stage's job as `notify_job`, so the chain is timed from the first click and emails once. GT Toolkit Tuning, which runs other tools' sweeps in sequence, times itself with `_start_notify_job()` and sets `_notify_outer_job` while its queue runs; any job created meanwhile is marked suppressed, so only the orchestrator emails.

- **Detached training** (`_build_notify_cfg()` → `_training_jobs.launch_detached(..., notify=...)`, §7): Tab 5's two launchers pass the SMTP settings to the supervisor script when the switch is on, with no threshold, so the email still arrives after napari has closed. Incomplete SMTP settings launch the run anyway, without email, and say so in the status line.

```
_finish_notify_job(job, ok, subject=None, body=""):     # any
thread, never raises
    if job already done: return                          # first
call wins (lock)
    mark done
    if job suppressed, or switch off, or elapsed < threshold, or
no recipient: return
    if SMTP username/password missing: print a note, return
    subject ← "[ZF-Microglia-AI] <label> finished|FAILED" unless
given
    body    ← job, status, start, end, duration + the job's own
text
    send via _training_jobs.send_notification_email in a new
daemon thread
        (errors only printed)
```

Worker threads never read Qt widgets: `_sync_notify_settings()` keeps a GUI-thread snapshot of the panel (`self._notify_settings`) updated on every edit, and `_finish_notify_job` reads only that. Fields are persisted on edit-finished (`_persist_notify_fields`, password via `_secrets.py`, never plaintext); the switch and threshold on change.

The recovery (autosave) timer

`_start_recovery_timer()` — the one `QTimer` in the plugin meant to live for the whole session (every other background-work `QTimer` calls `deleteLater()` once its job finishes, so its closure — often a full label-array copy — isn't pinned in memory for the rest of the session). Its interval comes from Tab 6's **Autosave Labels Layer** (default 30

minutes; minutes or hours; `_current_recovery_interval_ms()`), and `_on_autosave_settings_changed()` persists the setting and restarts the timer so a new interval takes effect immediately. Each tick, `_do_recovery_save()` does nothing if the checkbox is off or no Labels layer is picked in Tab 3's shared selector; otherwise it saves **only that one layer** to `<output_dir>/<layer_name>_recovery.tif` (int32) in a background thread, overwriting the previous save — a crash loses at most one interval of manual label editing.

`_refresh_*` family — keeping the UI in sync with the viewer

Each wired to `viewer.layers.events.inserted`/`.removed` (and, for the active-selection-dependent ones, `.selection.events.changed`), so the UI never goes stale as layers are added/removed/renamed elsewhere: - **`_refresh_layer_info()`** / **`_update_labels_section_visibility()`**: shows/hides the Pixel Classifier vs. Cellpose-SAM Segmentation groups based on whether a matching `_NoBG`/`_ExtRm`-suffixed Image layer *exists anywhere in the viewer* — not just whether it's the currently active layer (a real historical bug: the old active-layer-only check could hide a route's tools the moment the user selected the Labels layer itself, exactly what "Use selected" requires). - **`_refresh_stats_layers()`**: repopulates every layer combo and runs the auto-fill rules above (§6). - **`_refresh_gt_layers()`**: repopulates Tab 5's GT Annotation Image combo (§7). - **`_refresh_meta_lbl()`**: updates the Z/Y/X µm + anisotropy display under Tab 1's Open button.

Handler index — every `_widget.py` function, by home section

Every button/combo handler in the file is documented, with real behavior, in the pipeline-stage section for the tab it belongs to — listed here so nothing in the file is left unaccounted for:

Tab	Section	Representative handlers
1 — Skin Remover	§3	`_on_open`, `_on_load_labels`, `_on_run`, `_on_browse_model`, `_on_bg_mode_changed`
2 — Create MG Labels	§4	`_on_create_labels`, `_on_run_cellpose_seg`, `_on_rerun_single_cell`, `_resolve_rerun_layers`, `_on_use_selected_label_rerun`,

Tab	Section	Representative handlers
3 — Edit MG Labels	§5	_on_browse_cp_model, _run_auto_correction_stage, _run_sanding_stage _on_resort_labels, _on_remove_debris, _on_split_label, _on_join_labels, _on_correct_label, _on_correct_adjacent_labels, _on_copy_label_to_adjacent_slice, _on_protect_skin, _on_remove_skin_label, _on_toggle_skin_visibility, _on_autocorrect_labels, _run_sanding_stage_ac, _on_save_labels, _on_use_selected_label_a/b, _on_correct_label_id_changed
4 — Statistics	§6	_on_generate_stats, _on_stats_backend_changed
5 — AI Tools	§7	_on_ai_tools_mode_changed, _on_gt_interpolate, _on_gt_generate_masks, _on_gt_image_changed, _on_prepare_monai_data, _on_mt_launch_training, _on_mt_stop_training, _resume_monai_job_if_active, _start_monai_polling, _on_ct_launch_training, _on_ct_stop_training, _resume_cellpose_job_if_active, _start_cellpose_polling, _on_ct_calib_run (branch_radius), _on_xz_run (XZYZ patches), _on_cc_run (combine crops), _write_cellpose_best_pointer
6 — Sweeps & Utilities	§8	_on_gtk_scan/_on_gtk_run/ _gtk_poll/_on_gtk_stop (GT Toolkit), _on_bs_run_sweep (brain/ MONAI), _on_ps_run_sweep (BG Threshold/Signal Erosion), _on_sg_run_sweep (Sigma),

Tab	Section	Representative handlers
		_on_kr_run_sweep (Cellprob + merge/min size/large-contact), _on_es_run_sweep/ _on_es_read_recommended (epoch sweep), _on_run_contrast_sweep (Calibrate Correct-Label Contrast), _on_gtscore_run (Score Against GT), _on_gtp_run (GT package), _on_send_test_email, _on_toggle_drift, _on_autosave_settings_changed
Shell (this section)	§9	__init__, _build_ui, _connect_signals, _status, _save_cfg, _update_gt_history, _update_merge_stats_history, _hang_watch_start/stop, _get_notify_creds, _build_notify_cfg, _sync_notify_settings, _persist_notify_fields, _new_notify_job, _start_notify_job, _job_thread, _finish_notify_job, _on_send_test_email, _start_recovery_timer, _output_dir, _get_layer_scale, _active_layer, _active_labels_layer, _edit_labels_layer, _refresh_*, _current_min_volume, _lo_sweep_sample_params, _on_ai_tools_mode_changed, _update_labels_section_visibility

10. Module reference (quick index)

File	Lines	Purpose	Guide section
_widget.py	12,052	Application shell — all 6 tabs, every button	§3–§9

File	Lines	Purpose	Guide section
		handler, config, layer resolution	
`_labeling.py`	3,281	Pixel Classifier connected-components engine + every label-editing primitive (resort, debris, split, join, correct, copy, skin protection)	§4, §5
`_auto_correction.py`	1,021	6-step unattended per-cell auto-correction pipeline, chained after Cellpose-SAM Segmentation	§5
`_cellpose_seg.py`	820	Cellpose-SAM `do_3D` inference + GMM cleanup + Krendl safe-merge + large-contact merge + final safety net	§4
`_statistics.py`	764	Per-label morphology/ spatial/intensity statistics + natural-language descriptions	§6
`_krendl_sweep.py`	842	Cellprob voxel-level GT sweep + real-prediction max_gap/ min_contact, Min size and Large-contact calibration	§8
`_pixel_sweep.py`	609	BG Threshold x Signal Erosion / Smooth σXY x σZ GT sweeps + GT-measured floors (`min_volume_from_gt`, `min_hole_size_from_gt`, `min_intercell_gap_um`)	§8
`_io.py`	284		§2

File	Lines	Purpose	Guide section
		Metadata-aware `.tif`/`.ims` loading, voxel-scale resolution	
`_grow_correct.py`	540	Auto-grow / until-stable orchestration around Correct Label's 2D and 3D engines	§5
`_training_jobs.py`	378	Cross-platform detached-process training job management (launch, poll, kill, patience early-stop, email)	§7
`_gt_annotation.py`	322	Polygon-based hand-drawn GT annotation (brain/skin masks)	§7
`_combine_crops.py`	85	Symlinks one fish's crop folder into a shared multi-fish training pool	§7
`_epoch_sweep.py`	275	GT-verified Cellpose-SAM checkpoint (epoch) sweep, bbox-restricted	§8
`_contrast_sweep.py`	321	Self-referential Correct-Label contrast (`lo`) calibration against Cellpose-SAM's own output	§5
`_background.py`	193	Population-mode background estimation + 3 background-removal modes	§3
`_xzyz_patches.py`	300	Orientation-slice (XY/XZ/YZ) 2D training-crop generation, the current Cellpose-SAM training methodology	§7

File	Lines	Purpose	Guide section
`_gt_toolkit.py`	248	Single-folder auto-discovery of a fish's files + step preconditions ("GT Toolkit Tuning Tool")	§8
`_gt_package.py`	209	Zips a GT-correction package (source image, correction starting point, guide, stats CSV) for external manual review	§8
`_brain_sweep.py`	165	GT-verified MONAI Threshold x Erosion sweep for the brain mask	§8
`_secrets.py`	188	Two-tier (OS keyring → local Fernet-encrypted file) secret storage	§2
`_crop_truncation.py`	146	Zeros out incidentally-truncated neighbor labels in training crops	§7
`_branch_calibration.py`	134	Measures real branch radius from GT to recalibrate the branch-weighted training loss	§7
`_gt_score.py`	126	Whole-fish Hungarian-matched TP/FP/FN/ Score/IoU/Dice scoring, behind Score Against GT	§8
`_live_progress.py`	116	Captures MONAI's/ Cellpose's own otherwise-invisible progress output into the GUI log	§2
`_sieve.py`	105	Generic coarse-to-fine sweep-narrowing engine, shared by	§2

File	Lines	Purpose	Guide section
		every continuous-axis GT sweep	
`_ai_tools.py`	89	argv builders + default script-path resolution for the 3 training scripts	§7
`_sanding.py`	209	Cavity fill + sigma-softening (contour polishing) pass, chained after any label-regenerating correction	§4
`_reader.py`	58	napari File-menu integration (`.tif`/`.ims` reader registration)	§2
`_gpu_check.py`	55	GPU/VRAM classification (informational banner, never a hard gate)	§2
`__main__.py`	53	CLI entry point / pre-loading a file at launch	—
`_inference.py`	169	MONAI 3D U-Net sliding-window brain inference	§3
`__init__.py`	22	Package version + widget export	—

(`training_scripts/` — `prepare_data.py`, `train.py`, `train_xzyz.py` — *ships inside the package as launchable subprocess scripts, not imported; see* `_ai_tools.py` *and §7.)*

End of Software Guide.